

DSA: Two Pointers, Sliding Window & Matrix - తెలుగు గైడ్ (LeetCode 150, SSE)

ఈ document చదివిన తర్వాత Two Pointers, Sliding Window, Matrix — ఈ మూడు patterns ని మళ్ళీ జీవితంలో మర్చిపోలేవు. DSA అస్సలు తెలియకపోయినా పర్లేదు: ప్రతి problem కి మొదట ఎలా ఆలోచించాలి (intuition), తర్వాత brute force నుండి optimal కి ఎలా వెళ్ళాలి, ప్రతి pattern కి real-life analogy, reusable template, dry run, complexity, గుర్తుంచుకోవాల్సినవి, సాధారణ తప్పులు — అన్నీ ఉంటాయి.

లక్ష్యం: SSE interview కి prepare అవుతూ, coding తెలిసినా DSA patterns కొత్త అయిన engineer ని — "problem చూడగానే ఏ pattern వాడాలో గుర్తుపట్టే" స్థాయికి తీసుకెళ్ళడం. "ఒకసారి చదివితే మర్చిపోకూడదు."

Note: Big-O, arrays, strings, hashing, recursion లాంటి absolute fundamentals

DSA_00_Foundations_Telugu.md లో ఉన్నాయి — ఆ basics ఇక్కడ మళ్ళీ చెప్పను, patterns మీద focus చేస్తాను. Solutions అన్న **JavaScript (ES2020+)** లో, runnable & commented.

విషయ సూచిక (Table of Contents)

Pattern Primer 1 — Two Pointers (opposite-ends vs same-direction/fast-slow)

1. Valid Palindrome (LeetCode #125) — Easy
2. Is Subsequence (LeetCode #392) — Easy
3. Two Sum II - Input Array Is Sorted (LeetCode #167) — Medium
4. Container With Most Water (LeetCode #11) — Medium
5. 3Sum (LeetCode #15) — Medium

Pattern Primer 2 — Sliding Window (fixed vs variable window, expand/shrink template)

6. Minimum Size Subarray Sum (LeetCode #209) — Medium
7. Longest Substring Without Repeating Characters (LeetCode #3) — Medium
8. Substring with Concatenation of All Words (LeetCode #30) — Hard
9. Minimum Window Substring (LeetCode #76) — Hard

Pattern Primer 3 — Matrix Traversal (boundaries, direction, in-place tricks)

10. Valid Sudoku (LeetCode #36) — Medium
11. Spiral Matrix (LeetCode #54) — Medium
12. Rotate Image (LeetCode #48) — Medium
13. Set Matrix Zeroes (LeetCode #73) — Medium
14. Game of Life (LeetCode #289) — Medium

Pattern: Two Pointers

వివరణ

Two Pointers అనేది array లేదా string మీద **రెండు index variables** (pointers) పెట్టి, వాటిని తెలివిగా కదిలిస్తూ సమస్యను solve చేసే technique. ఒకే loop, ఒకే pass లో పని అవుతుంది – అందుకే చాలా సందర్భాల్లో ఇది **$O(n^2)$ brute force** ని **$O(n)$ కి తగ్గిస్తుంది.**

రెండు ప్రధాన variants ఉన్నాయి – ఈ తేడా గుర్తుంచుకుంటే సగం యుద్ధం గెలిచినట్టే:

- 1. Opposite-ends (converging pointers) – రెండు చివరల నుండి మధ్యకి:** `left = 0`, `right = n-1` పెట్టి, ఒక condition ఆధారంగా `left++` లేదా `right--` చేస్తూ మధ్యలో కలిసేదాకా వెళ్తాం (`while (left < right)`). ఇది ఎక్కువగా **sorted array** లేదా **palindrome** లాంటి symmetric సమస్యలకి. కీలకం: array sort అయ్యుంటే, `left` ని కదిపితే sum/value పెరుగుతుంది, `right` ని కదిపితే తగ్గుతుంది – ఈ monotonic property వల్ల ఏ pointer కదపాలో decide చేయగలం.
- 2. Same-direction (fast-slow pointers) – రెండూ ఒకే వైపు, వేర్వేరు వేగంతో:** `slow` నెమ్మదిగా, `fast` వేగంగా – రెండూ ఎడమ నుండి కుడికి. `slow` "ఇప్పటిదాకా ఖరారైన మంచి భాగం" ఎక్కడ ముగిసిందో గుర్తుపెడుతుంది, `fast` ముందుకు వెళ్ళి explore చేస్తుంది. ఇది **in-place array modification** (duplicates తీసేయడం), **subsequence check**, **linked list cycle detection** (Floyd's) లాంటి వాటికి.

Real-life Scenario

Opposite-ends = dictionary లో word వెతకడం. Dictionary sorted గా ఉంది. మీరు ఒక చేయి మొదటి page మీద, ఇంకో చేయి చివరి page మీద పెట్టి – మీ word ఏ వైపు ఉందో బట్టి ఒక చేయి లోపలికి కదిలిస్తారు. రెండు చేతులూ దగ్గరవుతూ target దగ్గరకి చేరతాయి. Linear గా page-by-page వెతకరు.

Fast-slow = train platform మీద చెత్త ఏరడం. ఒకరు (fast) platform అంతా నడుస్తూ ప్రతి వస్తువునీ చూస్తారు; ఇంకొకరు (slow) వెనకే వస్తూ, fast "ఇది మంచిది, ఉంచు" అన్న వస్తువుల్ని మాత్రమే వరుసగా పేర్చుకుంటూ వెళ్తారు. చివరికి slow దగ్గర శుభ్రమైన వరుస తయారవుతుంది.

ఎప్పుడు వాడాలి / ఎలా గుర్తించాలి

ఈ **keywords / signals** కనిపిస్తే two pointers ఆలోచించు:

- **"sorted array"** + "find a pair / triplet / target sum" → opposite-ends. (Sorting monotonic property ఇస్తుంది.)
- **"palindrome"**, "reverse", "compare from both ends" → opposite-ends.
- **"in-place"**, "without extra space / $O(1)$ space", "remove / move elements" → fast-slow (same-direction).
- **"maximize / minimize"** something between two indices (area, distance) → opposite-ends greedy.
- Brute force $O(n^2)$ nested loop కనిపిస్తే, "రెండో loop బదులు ఇంకో pointer పెట్టొచ్చా?" అని ప్రశ్నించు.

గుర్తుంచుకో: Two pointers వాడాలంటే సాధారణంగా data కి ఏదో **order / structure** (sorted, లేదా symmetric) ఉండాలి. Random unsorted data లో pair వెతకడానికి hashing బాగుంటుంది, two pointers కాదు.

Reusable Template (JavaScript)

```
// Variant 1: Opposite-ends (converging) – sorted array / palindrome
function opposite(arr, target) {
  let left = 0, right = arr.length - 1;
  while (left < right) {
    const sum = arr[left] + arr[right];
    if (sum === target) return [left, right]; // దొరికింది
    else if (sum < target) left++; // ఇంకా పెద్దది కావాలి → left కుడికి
    else right--; // ఇంకా చిన్నది కావాలి → right ఎడమకి
  }
  return [-1, -1];
}

// Variant 2: Same-direction (fast-slow) – in-place filter
function sameDirection(arr, keep) {
  let slow = 0; // ఖరారైన మంచి భాగం చివర
  for (let fast = 0; fast < arr.length; fast++) {
    if (keep(arr[fast])) { // fast ప్రతిదీ చూస్తుంది
      arr[slow] = arr[fast]; // మంచిది అయితేనే slow దగ్గర పెట్టు
      slow++;
    }
  }
  return slow; // కొత్త length (మంచి elements సంఖ్య)
}
```

Complexity

- **Time:** సాధారణంగా $O(n)$ – ప్రతి pointer array ని ఒక్కసారే చాటుతుంది (input sorted కాకపోతే sort కి $O(n \log n)$ కలుస్తుంది).
- **Space:** $O(1)$ – కొన్ని index variables తప్ప extra memory అవసరం లేదు. ఇదే two pointers యొక్క అసలు బలం.

1. Valid Palindrome (LeetCode #125) – Easy

సమస్య: ఒక string s ఇచ్చారు. దానిలో **alphanumeric characters** (a-z, A-Z, 0-9) మాత్రమే పరిగణించి, **case ని ignore చేసి**, అది **palindrome** (ముందు నుండి చదివినా, వెనుక నుండి చదివినా ఒకటే) అవునో కాదో చెప్పు. Spaces, punctuation, symbols అన్నీ వదిలేయాలి. Constraints: $1 \leq s.length \leq 2 \cdot 10^5$, s లో printable ASCII characters ఉంటాయి.

ఉదాహరణ:

- Input: $s = \text{"A man, a plan, a canal: Panama"}$ → Output: **true** (శుభ్రం చేస్తే amanaplanacanalpanama – palindrome).
- Input: $s = \text{"race a car"}$ → Output: **false** (raceacar – palindrome కాదు).
- Input: $s = \text{" "}$ → Output: **true** (alphanumeric ఏదీ లేదు → ఖాళీ string ని palindrome గా పరిగణిస్తాం).

ఎలా ఆలోచించాలి:

1. Palindrome అంటే ఏమిటి? మొదటి character = చివరి character, రెండో = చివరి-నుండి-రెండో... ఇలా **రెండు చివరల నుండి లోపలికి** compare చేసుకుంటూ వెళ్తే, అన్నీ match అయితే palindrome.
2. అంటే సహజంగానే **opposite-ends two pointers!** **left** మొదట్లో, **right** చివర్లో.
3. కానీ ఇక్కడ ఒక మెలిక: spaces, commas, colons వదిలేయాలి. కాబట్టి compare చేసే ముందు — **left** alphanumeric కాకపోతే కుడికి జరుపు, **right** alphanumeric కాకపోతే ఎడమకి జరుపు. **ఏం గమనించాలి?** "శుభ్రం చేయడం" (cleaning) ని ప్రత్యేక step గా కాకుండా, compare చేస్తూనే on-the-fly గా చేయొచ్చు.
4. Case ignore — compare చేసేటప్పుడు రెండింటినీ **toLowerCase()** చేసి చూడు.

Brute Force:

```
function isPalindromeBrute(s) {
  // step 1: alphanumeric matrame unchi, lowercase ki marchi kotta string build cheyyi
  const cleaned = s.toLowerCase().replace(/^[a-z0-9]/g, "");
  // step 2: reverse chesi compare cheyyi
  const reversed = cleaned.split("").reverse().join("");
  return cleaned === reversed;
}
```

Time **O(n)**, Space **O(n)** — cleaned + reversed రెండు కొత్త strings తయారవుతాయి. ఇది పని చేస్తుంది కానీ **extra O(n) memory** వాడుతుంది. Interviewer "O(1) space లో చేయగలవా?" అని అడిగితే ఇది సరిపోదు.

Optimal Approach:

- **Insight:** కొత్త string అస్సలు తయారు చేయనవసరం లేదు. Original string మీదే రెండు pointers పెట్టి, non-alphanumeric వాటినీ skip చేస్తూ compare చేస్తే చాలు.
- **Pattern:** Two Pointers — opposite-ends (converging).
- **Plan:** left = 0, right = n-1. **left < right** వరకు: (a) **left** alphanumeric అయ్యేదాకా **left++**, (b) **right** alphanumeric అయ్యేదాకా **right--**, (c) **s[left]** vs **s[right]** lowercase లో compare — తేడా వస్తే **false**, (d) **left++**, **right--**.

Solution (JavaScript):

```
function isPalindrome(s) {
  // helper: character alphanumeric aa? (a-z, A-Z, 0-9) — case-insensitive regex
  const isAlnum = (c) => /[a-z0-9]/i.test(c);

  let left = 0, right = s.length - 1;

  while (left < right) {
    // left vaipu nunchi non-alphanumeric anni skip cheyyi
    while (left < right && !isAlnum(s[left])) left++;
    // right vaipu nunchi non-alphanumeric anni skip cheyyi
    while (left < right && !isAlnum(s[right])) right--;

    // ippudu rendu alphanumeric characters — lowercase lo compare
    if (s[left].toLowerCase() !== s[right].toLowerCase()) {
      return false; // teda vachhindi → palindrome kaadu
    }
    left++; // rendu vaipulaki lopaliki jarugu
  }
}
```

```

    right--;
  }
  return true; // anni match ayyayi → palindrome
}

// test
console.log(isPalindrome("A man, a plan, a canal: Panama")); // true
console.log(isPalindrome("race a car")); // false
console.log(isPalindrome(" ")); // true

```

Dry Run: s = "race a car" (indices: r0 a1 c2 e3 _4 a5 _6 c7 a8 r9)

అడుగు	left	right	s[left]	s[right]	ఏం జరిగింది
1	0	9	r	r	match → left=1, right=8
2	1	8	a	a	match → left=2, right=7
3	2	7	c	c	match → left=3, right=6
4	3	6	e	_ (space)	right non-alnum → right=5 (a)
5	3	5	e	a	e ≠ a → return false

raceacar లో మధ్యలో e కి ఎదురుగా a వచ్చింది → palindrome కాదు. సరైన సమాధానం.

Complexity:

- **Time: O(n)** — రెండు pointers కలిపి string ని ఒక్కసారే దాటుతాయి. Inner **while** loops కూడా మొత్తంగా n steps మించవు (amortized).
- **Space: O(1)** — కొత్త string లేదు, కేవలం రెండు index variables. Brute force O(n) space ని O(1) కి తగ్గించాం.

గుర్తుంచుకోవాల్సినది: "compare from both ends", "palindrome", "reverse and check" — ఈ మాటలు వినగానే **opposite-ends two pointers**. కొత్త copy తయారు చేసే బదులు in-place గా pointers కదిలిస్తే space O(1). ఈ skip-then-compare నమూనా చాలా string problems లో పనికొస్తుంది.

సాధారణ తప్పులు:

- Inner **while** loops లో **left < right** condition మర్చిపోవడం → అన్నీ non-alphanumeric అయిన string (",.!") లో pointer array బయటికి పోతుంది (out of bounds).
- Case ignore మర్చిపోవడం — 'A' vs 'a' ని వేరుగా చూసి తప్పు answer.
- ఒక వైపు skip చేయడం (**left** skip చేసి **right** skip మర్చిపోవడం).
- ఖాళీ / అన్నీ symbols ఉన్న string ని handle చేయకపోవడం — ఇది **true** కావాలి (vacuously palindrome).

2. Is Subsequence (LeetCode #392) — Easy

సమస్య: రెండు strings s మరియు t ఇచ్చారు. s అనేది t యొక్క **subsequence** అవునో కాదో చెప్పు. Subsequence అంటే — t లోని కొన్ని characters ని (సున్నా లేదా అంతకంటే ఎక్కువ) తీసేసి, **మిగిలిన వాటి వరుస (relative order)**

మార్పుకుండా s ని తయారు చేయగలగడం. Constraints: $0 \leq s.length \leq 100$, $0 \leq t.length \leq 10^4$, రెండూ lowercase English letters.

ఉదాహరణ:

- $s = "abc"$, $t = "ahbgdc"$ → **true** (a_b_c – a, b, c వరుసలోనే ఉన్నాయి).
- $s = "axc"$, $t = "ahbgdc"$ → **false** (x అనేది t లో లేదు).
- $s = ""$, $t = "abc"$ → **true** (ఖాళీ string అన్నిటికీ subsequence).

గమనిక: Subsequence $\neq$ Substring. Substring అంటే **వరుసగా (contiguous)** ఉండాలి; subsequence అంటే వరుస (order) కాపాడితే చాలు, మధ్యలో gaps ఉండొచ్చు.

ఎలా ఆలోచించాలి:

- s లోని మొదటి character ని t లో ఎడమ నుండి వెతుకు. దొరికాక, s లోని రెండో character ని – **అదే చోటి నుండి ముందుకి** వెతుకు (వెనక్కి కాదు, order కాపాడాలి కాబట్టి).
- అంటే రెండు pointers – ఒకటి s కి (i), ఇంకోటి t కి (j) – **రెండూ ఒకే వైపు (same-direction)** ముందుకి కదులుతాయి.
- t ని ఒక్కో character దాటుతూ ($j++$) వెళ్తాం. $t[j]$ అనేది ప్రస్తుతం మనకి కావలసిన $s[i]$ కి సమానమైతే – ఆ character దొరికినట్టు, $i++$. కాకపోతే j మాత్రం ముందుకి.
- ఏం గమనించాలి?** i ఎప్పుడూ ముందుకే వెళ్తుంది, వెనక్కి రాదు. i అనేది s పొడవుకి చేరితే – s మొత్తం t లో order లో దొరికినట్టు → **true**.

Brute Force: ప్రతి $s[i]$ కి t లో recursion / nested search చేయడం – worst case బాగా ఖరీదు (backtracking లో $O(2^n)$ దాకా). రెండు simple pointers ఉండగా అది అనవసరం.

Optimal Approach:

- Insight:** Greedy గా – $s[i]$ కి t లో **మొదటి match** వాడేయడం ఎప్పుడూ safe. దాన్ని వదిలి తర్వాతి match కోసం ఎదురుచూడాల్సిన అవసరం లేదు (తర్వాత match ఉంటే, ముందు match వాడినా order చెడదు).
- Pattern:** Two Pointers – same-direction.
- Plan:** $i = 0$ (s కి). t మీద j loop. $s[i] === t[j]$ అయితే $i++$. చివర్లో $i === s.length$ అయితే **true**.

Solution (JavaScript):

```
function isSubsequence(s, t) {
    let i = 0; // s lo mana pointer (ippudu edi kavalo)

    // t ni edama nunchi kudiki daatutunnam
    for (let j = 0; j < t.length && i < s.length; j++) {
        if (s[i] === t[j]) {
            i++; // s[i] dorikindi → tarvati character kosam
        }
        // match kakapote j matrame mundhuki (loop lo), i alage untundi
    }

    return i === s.length; // s motham dorikindaa?
}
```

```
// test
console.log(isSubsequence("abc", "ahbgdc")); // true
console.log(isSubsequence("axc", "ahbgdc")); // false
console.log(isSubsequence("", "abc")); // true
```

Dry Run: `s = "abc"` , `t = "ahbgdc"`

j	t[j]	కావలసినది s[i]	match?	i (తర్వాత)
0	a	a (i=0)	✓	1
1	h	b (i=1)	✗	1
2	b	b (i=1)	✓	2
3	g	c (i=2)	✗	2
4	d	c (i=2)	✗	2
5	c	c (i=2)	✓	3

చివర్లో `i = 3 = s.length` → `true` . `s` మొత్తం order లో దొరికింది.

Complexity:

- **Time: $O(n)$** – ఇక్కడ `n = t.length` . `t` ని ఒక్కసారే దాటుతాం. (`s` కంటే `t` పెద్దది కాబట్టి dominant term `t` .)
- **Space: $O(1)$** – రెండు index variables మాత్రమే.

గుర్తుంచుకోవాల్సినది: "order కాపాడుతూ ఒక sequence ఇంకో sequence లో ఉందా?" అనే ప్రశ్న = **same-direction two pointers**. ఒక pointer "కావలసిన pattern" మీద, ఇంకోటి "పెద్ద data" మీద – pattern match అయినప్పుడే pattern pointer ముందుకి. Merge, matching లాంటి చాలా problems ఇదే idea.

Follow-up (interview లో అడుగుతారు): "కోట్ల `s` లని ఒకే `t` కి check చేయాలంటే?" – అప్పుడు `t` ని ఒకసారి preprocess చేసి, ప్రతి character కి దాని positions list (sorted indices) ని map లో store చేసి, ప్రతి `s` కి binary search తో check చేస్తాం. అప్పుడు ప్రతి `s` కి $O(|s| \cdot \log|t|)$. ఈ follow-up చెప్తే seniority కనిపిస్తుంది.

సాధారణ తప్పులు:

- `i < s.length` condition loop లో పెట్టకపోవడం → `s` అయిపోయాక కూడా అనవసరంగా `t` దాటడం (correctness సరే కానీ inefficient; పైగా `s[i]` undefined తో compare).
- Substring అనుకొని contiguous match వెతకడం – subsequence కి gaps allowed.
- ఖాళీ `s` కి `false` return చేయడం – ఖాళీ string ఎప్పుడూ subsequence (`true`).

3. Two Sum II - Input Array Is Sorted (LeetCode #167) – Medium

సమస్య: నాన్-డిక్రీజింగ్ order లో sort అయిన (ascending) array numbers (1-indexed) మరియు ఒక `target` ఇచ్చారు. కలిపితే `target` వచ్చే సరిగ్గా రెండు numbers ని కనిపెట్టి, వాటి 1-based indices [`index1`, `index2`] (`index1 < index2`) return చెయ్యి. సమాధానం సరిగ్గా ఒక్కటే ఉంటుంది. ఒకే element ని రెండుసార్లు వాడకూడదు. **$O(1)$**

extra space వాడాలి (constraint!). Constraints: $2 \leq \text{numbers.length} \leq 3 \cdot 10^4$, $-1000 \leq \text{numbers}[i] \leq 1000$, sorted.

ఉదాహరణ:

- `numbers = [2,7,11,15]` , `target = 9` → `[1,2]` ($2 + 7 = 9$).
- `numbers = [2,3,4]` , `target = 6` → `[1,3]` ($2 + 4 = 6$).
- `numbers = [-1,0]` , `target = -1` → `[1,2]` .

ఎలా ఆలోచించాలి:

- మామూలు Two Sum (unsorted) లో hash map వాడతాం — $O(n)$ time, $O(n)$ space. కానీ ఇక్కడ **"sorted" + $O(1)$ space** అని ప్రత్యేకంగా చెప్పారు. ఇవి రెండూ two pointers కి సూటి సంకేతాలు (dead giveaway).
- Array sorted కాబట్టి — ఒక అద్భుతమైన property వాడొచ్చు: **ఎడమ చివర అతిచిన్న number, కుడి చివర అతిపెద్ద number**. `left` ని కుడికి కదిపితే sum పెరుగుతుంది, `right` ని ఎడమకి కదిపితే sum తగ్గుతుంది. ఈ **monotonic** ప్రవర్తనే మనకి దిక్సూచి.
- `left = 0` , `right = n-1` పెట్టి `sum = numbers[left] + numbers[right]` చూడు:
 - `sum === target` → దొరికింది!
 - `sum < target` → చిన్నదైంది, పెంచాలి → `left++` (పెద్ద number కావాలి).
 - `sum > target` → పెద్దదైంది, తగ్గించాలి → `right--` (చిన్న number కావాలి).
- ఏం గమనించాలి — ఇది ఎందుకు safe?** `sum > target` అయ్యి `right--` చేసినప్పుడు, ప్రస్తుత `right` ని ఏ ఎడమ `element` తో కలిపినా (అన్నీ `numbers[left]` కంటే $\geq$) sum ఇంకా పెద్దదే అవుతుంది. కాబట్టి ఈ `right` ఏ solution లోనూ ఉండదు — దాన్ని శాశ్వతంగా వదిలేయొచ్చు. ప్రతి అడుగు ఒక candidate ని కచ్చితంగా తొలగిస్తుంది.

Brute Force:

```
function twoSumBrute(numbers, target) {
  for (let i = 0; i < numbers.length; i++) {
    for (let j = i + 1; j < numbers.length; j++) {
      if (numbers[i] + numbers[j] === target) return [i + 1, j + 1];
    }
  }
}
```

Time $O(n^2)$, Space $O(1)$. ప్రతి jodi ని try చేస్తుంది. Sorted అనే విలువైన సమాచారాన్ని అస్సలు వాడలేదు — అందుకే నెమ్మది.

Optimal Approach:

- Insight:** Sorted array యొక్క monotonic property వల్ల, ప్రతి comparison తర్వాత రెండు వైపులా ఏ pointer కదపాలో కచ్చితంగా తెలుస్తుంది — దాంతో $O(n^2) \rightarrow O(n)$.
- Pattern:** Two Pointers — opposite-ends (converging).
- Plan:** పైన చెప్పిన మూడు cases తో `while (left < right)` .

Solution (JavaScript):

```
function twoSum(numbers, target) {
  let left = 0, right = numbers.length - 1;

  while (left < right) {
    const sum = numbers[left] + numbers[right];

    if (sum === target) {
      return [left + 1, right + 1]; // 1-indexed kaabatti +1
    } else if (sum < target) {
      left++; // sum penchali → pedda number vaipuki
    } else {
      right--; // sum taggincali → chinna number vaipuki
    }
  }

  return []; // problem prakaram ikkadiki raadu
}

// test
console.log(twoSum([2, 7, 11, 15], 9)); // [1, 2]
console.log(twoSum([2, 3, 4], 6)); // [1, 3]
console.log(twoSum([-1, 0], -1)); // [1, 2]
```

Dry Run: numbers = [2,7,11,15] , target = 9

left	right	numbers[left]	numbers[right]	sum	vs target	చర్య
0	3	2	15	17	> 9	right-- → 2
0	2	2	11	13	> 9	right-- → 1
0	1	2	7	9	= 9	return [1, 2]

రెండే comparisons లో (nested loop లో 3+ కావలసినది) సమాధానం. 1-based కాబట్టి indices 0,1 → [1,2] .

Complexity:

- **Time: O(n)** – left, right కలిపి array ని ఒక్కసారే scan చేస్తాయి (ప్రతి iteration లో ఏదో ఒక pointer కదులుతుంది, అవి ఎప్పటికీ దాటవు).
- **Space: O(1)** – hash map లేదు, రెండు pointers మాత్రమే. Constraint తీరింది. (Hash-map version O(n) space అవుతుంది.)

గుర్తుంచుకోవాల్సినది: "Sorted array + pair with target sum + O(1) space" = opposite-ends two pointers, ఇది reflex లా రావాలి. Sorting ఇచ్చే monotonic property వల్ల ప్రతి అడుగులో ఒక pointer ని కచ్చితంగా కదపగలం. Array sorted కాకపోతే – hashing (O(n) space) మేలు, లేదా ముందు sort చేసుకోవాలి (అది 3Sum లో వాడతాం).

సాధారణ తప్పులు:

- **1-indexed** మర్చిపోయి +1 చేయకపోవడం – ఈ problem లో అత్యంత common తప్పు.
- Unsorted array కి ఇదే two pointers వాడటం – sort లేకపోతే monotonic property లేదు, answer తప్పు.
- **left <= right** రాయడం (< బదులు) – ఒకే element ని రెండుసార్లు వాడే ప్రమాదం.
- Overflow (JS లో పెద్ద numbers) – ఇక్కడ constraints చిన్నవి కాబట్టి సమస్య లేదు, కానీ ఇతర భాషల్లో గుర్తుంచుకోవాలి.

4. Container With Most Water (LeetCode #11) — Medium

సమస్య: height అనే array ఇచ్చారు — ఇందులో ప్రతి height[i] అనేది x-axis మీద **i** దగ్గర నిలబెట్టిన ఒక నిలువు గీత (vertical line) ఎత్తు. ఏదైనా రెండు గీతలు + x-axis కలిసి ఒక container (తొట్టి) తయారు చేస్తాయి. అందులో పట్టే అత్యధిక నీటి పరిమాణం (area) కనిపెట్టు. తొట్టిని ఒంచకూడదు (గీతలు నిలువుగానే). Constraints: $2 \leq \text{height.length} \leq 10^5$, $0 \leq \text{height}[i] \leq 10^4$.

Area యొక్క formula: రెండు గీతల మధ్య దూరం (width) × అందులో పొట్టి గీత ఎత్తు (నీళ్ళు పొట్టి గీత మీదుగా పొంగిపోతాయి కదా): $\text{area} = (\text{right} - \text{left}) \times \min(\text{height}[\text{left}], \text{height}[\text{right}])$.

ఉదాహరణ:

- height = [1,8,6,2,5,4,8,3,7] → 49 (index 1 (ఎత్తు 8) మరియు index 8 (ఎత్తు 7): width 7 × min(8,7)=7 → 49).
- height = [1,1] → 1.

ఎలా ఆలోచించాలి:

- Area రెండింటి మీద ఆధారపడుతుంది: **width** (దూరం) మరియు **height** (పొట్టి గీత). రెండూ ఎక్కువగా ఉంటే area ఎక్కువ. కానీ ఇవి కొన్నిసార్లు వ్యతిరేకంగా ఉంటాయి — దూరంగా ఉన్న గీతలు పొట్టివి కావచ్చు.
- అతిపెద్ద width ఏది?** మొదటి, చివరి గీతలు (left=0, right=n-1). కాబట్టి **అత్యంత వెడల్పు నుండి మొదలుపెడదాం** — ఇది opposite-ends two pointers కి సహజ ప్రారంభం.
- ఇప్పుడు లోపలికి కదలాలి (width తగ్గుతుంది). **ఏ pointer కదపాలి?** ఇక్కడే అసలు insight: area ని పొట్టి గీత limit చేస్తోంది. పొడవు గీతని కదిపితే — width తగ్గుతుంది, min ఇంకా ఆ పొట్టి గీత చేతిలోనే (పెరగదు) → area ఖచ్చితంగా తగ్గుతుంది/సమానం, ఎప్పటికీ పెరగదు. కాబట్టి **ఎప్పుడూ పొట్టి గీతని లోపలికి కదపాలి** — bottleneck అదే, దాన్ని మార్చితేనే మెరుగుపడే అవకాశం.
- ఏం గమనించాలి?** పొట్టి గీతని వదిలేయడం వల్ల మనం ఏ మంచి answer నీ కోల్పోము — ఆ పొట్టి గీత ప్రస్తుత widthలోనే గరిష్ఠం ఇచ్చింది; ఇంకా లోపలికి వెళ్తే width ఇంకా తగ్గుతుంది కాబట్టి అదే గీతతో మెరుగైనది రాదు.

Brute Force:

```
function maxAreaBrute(height) {
  let max = 0;
  for (let i = 0; i < height.length; i++) {
    for (let j = i + 1; j < height.length; j++) {
      const area = (j - i) * Math.min(height[i], height[j]);
      max = Math.max(max, area);
    }
  }
  return max;
}
```

Time $O(n^2)$, Space $O(1)$. అన్ని జతల్ని చూస్తుంది. $n = 10^5$ కి 10^{10} operations → TLE (Time Limit Exceeded). సరిపోదు.

Optimal Approach:

- **Insight:** అత్యధిక width తో మొదలుపెట్టి, ప్రతిసారి **పొట్టి గీతని** లోపలికి కదుపు. పొడవు గీతని కదపడం వల్ల area పెరిగే అవకాశమే లేదు కాబట్టి, ఆ దిశని పూర్తిగా వదిలేయొచ్చు.
- **Pattern:** Two Pointers – opposite-ends, greedy.
- **Plan:** left=0, right=n-1, max=0. ప్రతి అడుగులో area లెక్కించి **max** update చేసి, పొట్టి గీత వైపు pointer కదుపు.

Solution (JavaScript):

```
function maxArea(height) {
  let left = 0, right = height.length - 1;
  let max = 0;

  while (left < right) {
    const h = Math.min(height[left], height[right]); // potti geeta ettu
    const w = right - left;                          // width
    max = Math.max(max, h * w);                       // ippati max tho poolchu

    // ELLAPPUDU potti geeta ni lopaliki jarupu (adi bottleneck)
    if (height[left] < height[right]) {
      left++;
    } else {
      right--;
    }
  }

  return max;
}

// test
console.log(maxArea([1, 8, 6, 2, 5, 4, 8, 3, 7])); // 49
console.log(maxArea([1, 1]));                     // 1
```

Dry Run: height = [1,8,6,2,5,4,8,3,7] (కీలక అడుగులు)

left	right	h=min	w	area	max	కదిలింది
0	8	min(1,7)=1	8	8	8	left++ (1 పొట్టి)
1	8	min(8,7)=7	7	49	49	right-- (7 పొట్టి)
1	7	min(8,3)=3	6	18	49	right--
1	6	min(8,8)=8	5	40	49	right-- (సమానం → right)
1	5	min(8,4)=4	4	16	49	right--
...	...	...	...	< 49	49	...

మొదటి రెండో అడుగులోనే 49 దొరికింది; మిగతా అడుగులు దాన్ని దాటలేదు. చివర్లో left == right → **49**.

Complexity:

- **Time: O(n)** – ప్రతి iteration లో ఒక pointer లోపలికి కదులుతుంది; కలిపి n అడుగులు మాత్రమే. O(n²) brute force ని O(n) కి తగ్గించాం.

- **Space: $O(1)$** – కొన్ని variables మాత్రమే.

గుర్తుంచుకోవాల్సినది: "రెండు indices మధ్య ఏదో **maximize/minimize** చేయాలి (area, distance)" + brute force $O(n^2)$ → **opposite-ends two pointers with a greedy move**. కీలకం: ఏ *pointer* కదిపితే *improve* అయ్యే అవకాశం ఉందో ఆలోచించడం – ఇక్కడ bottleneck (పొట్టి గీత) ని కదిపితేనే మేలు. ఈ "bottleneck ని కదుపు" ఆలోచన చాలా greedy two-pointer problems లో వస్తుంది.

సాధారణ తప్పులు:

- పొడవు గీతని కదపడం (తప్పు దిశ) → కొన్ని cases లో సరైన answer miss.
- రెండు గీతలూ సమానం అయినప్పుడు ఏ వైపు కదిపినా ఒకటే (ఇక్కడ కంగారు అనవసరం) – కానీ కదపడం మాత్రం మర్చిపోకూడదు (infinite loop).
- $\text{Area} = \text{width} \times \min(\dots)$ అని కాకుండా pointer values ని కలపడం (ఇది two-sum కాదు, area problem).
- పొట్టి గీత ఎత్తు వాడాలి, పొడవుది కాదు – నీళ్ళు పొట్టి గీత మీదుగా పొంగుతాయి.

5. 3Sum (LeetCode #15) – Medium

సమస్య: nums array ఇచ్చారు. $\text{nums}[i] + \text{nums}[j] + \text{nums}[k] == 0$ అయ్యే **అన్ని విభిన్న triplets** $[\text{nums}[i], \text{nums}[j], \text{nums}[k]]$ ని return చెయ్యి (i, j, k వేర్వేరు indices). **Duplicate triplets ఉండకూడదు.** క్రమం (order) ముఖ్యం కాదు. Constraints: $3 \leq \text{nums.length} \leq 3000$, $-10^5 \leq \text{nums}[i] \leq 10^5$.

ఉదాహరణ:

- $\text{nums} = [-1, 0, 1, 2, -1, -4] \rightarrow [[-1, -1, 2], [-1, 0, 1]]$.
- $\text{nums} = [0, 1, 1] \rightarrow []$ (0 కి కలిపే triplet లేదు).
- $\text{nums} = [0, 0, 0] \rightarrow [[0, 0, 0]]$.

ఎలా ఆలోచించాలి:

1. మూడు numbers కలిపి 0 కావాలి. Brute force – మూడు nested loops, $O(n^3)$. చాలా ఖరీదు.
2. చిన్న ఉపాయం: $a + b + c = 0$ అంటే $b + c = -a$. అంటే **మొదటి number a ని fix చేస్తే**, మిగతా రెండింటి కోసం $\text{target} = -a$ తో ఇది **Two Sum అవుతుంది!** (Problem 3 లో చేసిందే.)
3. Two Sum ని $O(n)$ two-pointer తో చేయాలంటే array **sorted** ఉండాలి. కాబట్టి ముందు nums ని sort చేద్దాం ($O(n \log n)$) – ఒక్కసారే, పర్లేదు).
4. ఇప్పుడు: ప్రతి i కి ($a = \text{nums}[i]$), దాని కుడివైపు భాగం మీద opposite-ends two pointers ($\text{left} = i+1$, $\text{right} = n-1$) నడిపి $-\text{nums}[i]$ ని వెతుకు.
5. **ఏం గమనించాలి – duplicates అసలు సమస్య:** $[-1, -1, 2]$ రెండుసార్లు రాకూడదు. Sorted array లో duplicates పక్కపక్కనే ఉంటాయి. కాబట్టి – (a) i దగ్గర, $\text{nums}[i] === \text{nums}[i-1]$ అయితే ఆ i ని **skip** (అదే starting value మళ్ళీ వద్దు); (b) match దొరికాక left , right లని కూడా వాటి duplicate విలువల మీదుగా జరిపి skip చేయాలి.
6. **Bonus optimization:** sorted కాబట్టి $\text{nums}[i] > 0$ అయితే – మిగతావన్నీ ఇంకా పెద్దవి (positive), sum 0 కాదు → **break**.

Brute Force: మూడు nested loops, ప్రతి triplet check, duplicates ని **Set** తో తీసేయడం. Time $O(n^3)$, Space $O(n)$.
n = 3000 కి $2.7 \cdot 10^{10}$ operations → చాలా నెమ్మది.

Optimal Approach:

- **Insight: Sort + Fix one + Two Pointers.** ఒక element fix చేసి మిగతా దాన్ని Two-Sum-on-sorted గా మార్చడం – ఇది "kSum" problems కి master pattern.
- **Pattern:** Two Pointers (opposite-ends), outer loop తో కలిపి.
- **Plan:** Sort → for i → duplicate-i skip → inner while (left < right) two-sum → match దగ్గర duplicate-left/right skip.

Solution (JavaScript):

```
function threeSum(nums) {
  nums.sort((a, b) => a - b); // ascending sort – two pointers ki avasaram
  const res = [];

  for (let i = 0; i < nums.length - 2; i++) {
    // 'i' duplicate aite skip (adhe modati value malli vaddu)
    if (i > 0 && nums[i] === nums[i - 1]) continue;
    // nums[i] positive aite migatavi anni pedda → sum 0 raadu, aagipo
    if (nums[i] > 0) break;

    let left = i + 1, right = nums.length - 1;
    const target = -nums[i]; // b + c = -a kaavali

    while (left < right) {
      const sum = nums[left] + nums[right];
      if (sum === target) {
        res.push([nums[i], nums[left], nums[right]]); // triplet dorikindi
        left++;
        right--;
        // left, right duplicates ni skip cheyyi (unique triplets kosam)
        while (left < right && nums[left] === nums[left - 1]) left++;
        while (left < right && nums[right] === nums[right + 1]) right--;
      } else if (sum < target) {
        left++; // penchali
      } else {
        right--; // taggincali
      }
    }
  }

  return res;
}

// test
console.log(threeSum([-1, 0, 1, 2, -1, -4])); // [[-1,-1,2],[-1,0,1]]
console.log(threeSum([0, 1, 1])); // []
console.log(threeSum([0, 0, 0])); // [[0,0,0]]
```

Dry Run: nums = [-1, 0, 1, 2, -1, -4] → sort → [-4, -1, -1, 0, 1, 2]

- i=0 (a=-4, target=4): left..right లో 1+2=3 max, 4 కి చేరదు → triplet లేదు.

- **i=1** (a=-1 , target=1): left=2(-1) , right=5(2) → sum 1 == target → **push [-1,-1,2]** . left→3, right→4 (duplicate skip ఏదీ లేదు). ఇప్పుడు left=3(0) , right=4(1) → sum 1 == target → **push [-1,0,1]** . left→4, right→3 → loop ముగిసింది.
- **i=2** (a=-1): nums[2] === nums[1] → **duplicate skip** ([-1,...] మళ్ళీ వద్దు).
- **i=3** (a=0 , target=0): left=4(1) , right=5(2) → sum 3 > 0 → right--; loop ముగిసింది.

Result: **[[-1,-1,2], [-1,0,1]]** . Duplicate [-1,-1,2] రాలేదు — skip logic పని చేసింది.

Complexity:

- **Time: $O(n^2)$** — sort $O(n \log n)$, తర్వాత outer loop $O(n) \times$ inner two-pointer $O(n) = O(n^2)$. Dominant term $O(n^2)$. $O(n^3)$ brute force కంటే చాలా మెరుగు.
- **Space: $O(1)$** అదనపు (output res ని లెక్కించకపోతే). In-place sort వాడాలి. (Sort recursion stack $O(\log n)$ అనుకోవచ్చు.)

గుర్తుంచుకోవాల్సినది: kSum master pattern — "k numbers కలిపి target" → sort చేసి, k-2 elements ని loops తో fix చేసి, చివరి రెండింటికి two-pointer. 3Sum, 4Sum, 3Sum Closest అన్నీ ఇదే. Unsorted lo pair/triplet-sum కనిపిస్తే **"ముందు sort చేస్తే two pointers వస్తుందా?"** అని ఆలోచించు. Duplicate handling ఈ pattern లో అత్యంత గమ్యుత్తు — sorted లో duplicates పక్కపక్కనే కాబట్టి skip సులభం.

సాధారణ తప్పులు:

- **Duplicate skip మర్చిపోవడం** — అదే triplet పలుసార్లు వస్తుంది (ఈ problem లో #1 తప్పు). **i** , **left** , **right** మూడు చోట్లా skip చేయాలి.
- Match దొరికాక **left++ / right--** చేయకపోవడం → infinite loop.
- Sort మర్చిపోవడం → two-pointer logic పూర్తిగా విఫలం.
- **nums[i] === nums[i-1]** బదులు **nums[i] === nums[i+1]** skip చేయడం — ఇది మొదటి occurrence నే skip చేసి valid triplets కోల్పోతుంది. ఎప్పుడూ **వెనక్కి (i-1)** చూసి skip చేయాలి.

Pattern: Sliding Window

వివరణ

Sliding Window అనేది two-pointers యొక్క ప్రత్యేక రూపం — ఇక్కడ రెండు pointers (**left** , **right**) ఒక **విండో (window)** యొక్క రెండు అంచులు. ఈ window అనేది array/string లోని ఒక **contiguous (వరుసగా ఉన్న) భాగం** [left, right] . Window ని ఎడమ నుండి కుడికి **జారుస్తూ (slide)** పోతూ, ప్రతి window గురించి కావలసిన సమాచారం (sum, count, distinct characters...) ని **incrementally** update చేసుకుంటాం — ప్రతిసారి మొదటి నుండి లెక్కించము. అదే దీని వేగానికి రహస్యం: **$O(n^2)/O(n^3)$ "అన్ని subarrays చూడటం" → $O(n)$.**

రెండు రకాలు:

1. Fixed-size window (స్థిర పరిమాణం k): Window size ఎప్పుడూ k . కుడి నుండి కొత్త element చేర్చినప్పుడు, window k దాటితే ఎడమ నుండి ఒకటి తీసేస్తాం — size స్థిరంగా ఉంటుంది. ఉదా. "size k subarray యొక్క max sum", "k రోజుల moving average".

2. Variable-size window (మారే పరిమాణం): Window ఒక **condition** ఆధారంగా పెరుగుతుంది/కుంచించుకుంటుంది.

రెండు ఉప-రకాలు — ఈ తేడా అర్థమైతే sliding window master అయినట్టే:

- **అత్యంత పొడవైన (longest/max) valid window:** **right** ని పెంచుతూ (expand) window ని పెద్దది చేస్తాం; condition **violate** అయినప్పుడు మాత్రమే **left** ని కదిపి (shrink) తిరిగి valid చేస్తాం. Shrink తర్వాత window size తో max update.
- **అత్యంత చిన్న (shortest/min) valid window:** **right** ని పెంచుతూ window valid అయ్యేదాకా వెళ్తాం; valid అయ్యాక — **valid గా ఉన్నంతవరకు left** ని కదిపి కుంచించి, ప్రతిసారి min update చేస్తాం.

Real-life Scenario

Fixed window = రైలు కిటికీ నుండి బయటకు చూడటం. కిటికీ వెడల్పు స్థిరం (k). రైలు కదులుతున్నప్పుడు — కుడివైపు కొత్త దృశ్యం లోపలికి వస్తుంది, ఎడమవైపు పాత దృశ్యం బయటికి పోతుంది. మీరు ఎప్పుడూ **అదే వెడల్పు** ఫ్రేమ్ చూస్తారు, కానీ content జారుతూ మారుతుంది.

Variable window = గుంపు ఫోటో తీయడం. అందరూ ఫ్రేమ్ లో పట్టేదాకా camera zoom out చేస్తారు (right expand). ఒక అపరిచితుడు (unwanted) ఫ్రేమ్ లోకి వస్తే — ఎడమ అంచుని లోపలికి జరిపి అతన్ని బయట పెడతారు (left shrink). లక్ష్యం: సరైన గుంపుని కవర్ చేసే అత్యంత పెద్ద లేదా అత్యంత చిన్న ఫ్రేమ్.

ఎప్పుడు వాడాలి / ఎలా గుర్తించాలి

ఈ **keywords / signals** = sliding window:

- **"contiguous subarray / substring"** — ఇదే అతిపెద్ద సంకేతం. (Subsequence అయితే కాదు — అది వేరే.)
- **"longest / shortest / maximum / minimum"** subarray/substring **with some property** ($\text{sum} \geq \text{target}$, k distinct characters, no repeats...).
- **"subarray of size k "** → fixed window.
- **"at most / exactly K distinct"**, "without repeating characters", "sum equals/at least".
- Brute force "అన్ని subarrays generate చేసి check" ($O(n^2)/O(n^3)$) కనిపిస్తే → window గా మార్చగలమా అని చూడు.

Two Pointers vs Sliding Window: రెండూ same-direction pointers. తేడా — Two Pointers సాధారణంగా ఒక *జత elements* మీద దృష్టి; Sliding Window మధ్యలో ఉన్న మొత్తం *contiguous భాగం* (window) మీద దృష్టి, దాని aggregate state (sum/count) ని maintain చేస్తుంది.

Reusable Template (JavaScript)

```
// Fixed-size window (size k)
function fixedWindow(arr, k) {
    let sum = 0, best = -Infinity;
    for (let right = 0; right < arr.length; right++) {
        sum += arr[right];                // kudi nunchi kotta element cherchu
        if (right >= k) {                 // window k daatinte
            sum -= arr[right - k];        // edama nunchi paatadi teseyyi
        }
        if (right >= k - 1) {             // window nindaka
            best = Math.max(best, sum);
        }
    }
    return best;
}
```

```

        best = Math.max(best, sum);
    }
}
return best;
}

// Variable-size window (longest valid window)
function variableWindow(s) {
    let left = 0, best = 0;
    const state = new Map(); // window lopali samacharam
    for (let right = 0; right < s.length; right++) {
        // 1) EXPAND: s[right] ni window loki cherchu
        state.set(s[right], (state.get(s[right]) || 0) + 1);

        // 2) SHRINK: condition violate ainanta varaku edama anchu jarupu
        while (/* window invalid? */ false) {
            state.set(s[left], state.get(s[left]) - 1);
            if (state.get(s[left]) === 0) state.delete(s[left]);
            left++;
        }

        // 3) UPDATE: ippudu window valid → answer update
        best = Math.max(best, right - left + 1);
    }
    return best;
}

```

Complexity

- **Time: $O(n)$** – nested `while` ఉన్నా, `right` n సార్లు మాత్రమే పెరుగుతుంది, `left` కూడా ఎక్కువలో ఎక్కువ n సార్లు మాత్రమే పెరుగుతుంది. ప్రతి element window లోకి ఒకసారి వచ్చి ఒకసారి వెళ్తుంది → **amortized $O(n)$** . (ఇది చాలామంది "nested loop = $O(n^2)$ " అని పొరపడే చోటు – కాదు!)
- **Space: $O(1)$ లేదా $O(k)$** – window state (Map/array) size మీద ఆధారపడి; alphabet fixed అయితే (26 అక్షరాలు) $O(1)$.

6. Minimum Size Subarray Sum (LeetCode #209) – Medium

సమస్య: positive integers array `nums` మరియు positive number `target` ఇచ్చారు. $\text{Sum} \geq \text{target}$ అయ్యే **అతి చిన్న (minimal length)** contiguous subarray యొక్క పొడవును return చెయ్యి. అలాంటి subarray లేకపోతే `0`.

Constraints: $1 \leq \text{target} \leq 10^9$, $1 \leq \text{nums.length} \leq 10^5$, $1 \leq \text{nums}[i] \leq 10^4$.

ఉదాహరణ:

- `target = 7`, `nums = [2,3,1,2,4,3]` → 2 (`[4,3]` sum 7, పొడవు 2 – అతి చిన్నది).
- `target = 4`, `nums = [1,4,4]` → 1 (`[4]`).
- `target = 11`, `nums = [1,1,1,1,1,1,1,1]` → 0 (మొత్తం sum 8 < 11).

ఎలా ఆలోచించాలి:

1. "Contiguous subarray" + "అతి చిన్న పొడవు" + "sum $\geq$ target" – ఇవి sliding window కి స్పష్టమైన సంకేతాలు. అందునా **shortest valid window** రకం.
2. అన్ని numbers **positive** – అంటే window కి element చేర్చితే sum **ఖచ్చితంగా పెరుగుతుంది**, తీసేస్తే **తగ్గుతుంది**. ఈ monotonic ప్రవర్తనే sliding window ని సాధ్యం చేస్తుంది. (ఒకవేళ negatives ఉంటే ఇది పని చేయదు – అప్పుడు prefix-sum + వేరే technique.)
3. **Plan:** **right** ని పెంచుతూ window కి element చేర్చి sum పెంచు. Sum $\geq$ **target** అయిన క్షణం – window valid. ఇప్పుడు **వలెనంత కుంచించు**: valid గా ఉన్నంతవరకు **left** ని కదిపి, ప్రతిసారి **right - left + 1** తో min update చేసి, sum తగ్గించు.
4. **ఏం గమనించాలి?** valid అయ్యాక వెంటనే shrink చేయడం వల్ల – ప్రతి **right** కి "ఈ right తో ముగిసే అతి చిన్న valid window" దొరుకుతుంది. అన్ని right ల మీద min తీస్తే global answer.

Brute Force:

```
function minSubArrayLenBrute(target, nums) {
  let min = Infinity;
  for (let i = 0; i < nums.length; i++) {
    let sum = 0;
    for (let j = i; j < nums.length; j++) {
      sum += nums[j];
      if (sum >= target) { min = Math.min(min, j - i + 1); break; }
    }
  }
  return min === Infinity ? 0 : min;
}
```

Time **$O(n^2)$** , Space **$O(1)$** . ప్రతి start **i** కి subarray విస్తరిస్తుంది. $n=10^5$ కి నెమ్మది.

Optimal Approach:

- **Insight:** positive numbers వల్ల window sum monotonic. Valid అయ్యాక కుంచించడం ద్వారా ప్రతి right కి కనిష్ట window ని $O(1)$ amortized లో పొందుతాం.
- **Pattern:** Sliding Window – variable, **shortest valid**.
- **Plan:** left=0, sum=0, min=Infinity. **right** loop: **sum** += nums[right] ; **while** (sum >= target) → min update, **sum** -= nums[left] , **left**++ .

Solution (JavaScript):

```
function minSubArrayLen(target, nums) {
  let left = 0, sum = 0, min = Infinity;

  for (let right = 0; right < nums.length; right++) {
    sum += nums[right]; // EXPAND: kudi element cherchu

    // SHRINK: sum inka target ki saripodaa? valid unnanta shrink chey
    while (sum >= target) {
      min = Math.min(min, right - left + 1); // ee valid window podavu
      sum -= nums[left]; // edama element teseyyi
      left++;
    }
  }
}
```

```

    }
    return min === Infinity ? 0 : min;    // eppudu valid kaakapote 0
}

// test
console.log(minSubArrayLen(7, [2, 3, 1, 2, 4, 3]));    // 2
console.log(minSubArrayLen(4, [1, 4, 4]));            // 1
console.log(minSubArrayLen(11, [1, 1, 1, 1, 1, 1, 1, 1]));    // 0

```

Dry Run: target = 7 , nums = [2,3,1,2,4,3]

right	nums[right]	sum (expand తర్వాత)	shrink?	window	min
0	2	2	2<7 → no	—	∞
1	3	5	5<7 → no	—	∞
2	1	6	6<7 → no	—	∞
3	2	8	8≥7 → min=4 ([2,3,1,2]), sum=6, left=1	[1..3]	4
4	4	10	≥7 → min=4 ([3,1,2,4]), sum=7, left=2; ≥7 → min=3 ([1,2,4]), sum=6, left=3		3
5	3	9	≥7 → min=3 ([2,4,3]), sum=7, left=4; ≥7 → min=2 ([4,3]), sum=3, left=5		2

చివరి answer **2** ([4,3]). Window ఎలా పెరిగి, valid అయ్యాక కుంచించి కనిష్టం కనిపెడుతోందో గమనించు.

Complexity:

- **Time: O(n)** — right n సార్లు, left కూడా మొత్తంగా n సార్లు కదులుతుంది (ప్రతి element ఒకసారి add, ఒకసారి remove). Nested while ఉన్నా amortized O(n).
- **Space: O(1)** — sum, min, రెండు pointers మాత్రమే.

గుర్తుంచుకోవాల్సినది: "shortest/minimum contiguous subarray with sum ≥ X" = variable sliding window, **valid అయ్యాక aggressive గా shrink**. గుర్తు: shortest → valid అయ్యాక shrink చేస్తూ min; longest → invalid అయితేనే shrink చేస్తూ max (తర్వాతి problem లో చూస్తాం). ఈ "expand-till-valid, then shrink" నమూనా min-window problems అన్నిటికీ.

సాధారణ తప్పులు:

- min initial విలువ 0 పెట్టడం — ఎప్పటికీ update కాదు. Infinity పెట్టి చివర్లో 0 కి మార్చాలి.
- Shrink loop లో min update ని sum తీసేసిన తర్వాత చేయడం — valid window పోయాక కొలుస్తాం, తప్పు. తీసేసే ముందు update చేయాలి.
- Negatives ఉన్న arrays కి ఇదే వాడటం — monotonic property లేదు, ఈ approach విఫలం (అది వేరే problem).
- "≥ target" ని "== target" గా చదవడం — sum సరిగ్గా target కావాలని లేదు, ≥ చాలు.

7. Longest Substring Without Repeating Characters (LeetCode #3) — Medium

సమస్య: ఒక string s ఇచ్చారు. పునరావృత characters (repeating) లేని అతి పొడవైన substring యొక్క పొడవును కనిపెట్టు. Constraints: $0 \leq s.length \leq 5 \cdot 10^4$, s లో English letters, digits, symbols, spaces ఉండొచ్చు.

ఉదాహరణ:

- $s = \text{"abcabcbb"} \rightarrow 3$ ("abc" — repeat లేని పొడవైనది).
- $s = \text{"bbbbbb"} \rightarrow 1$ ("b").
- $s = \text{"pwwkew"} \rightarrow 3$ ("wke"; గమనిక "pwke" కాదు — అది substring కాదు, w రెండుసార్లు వస్తుంది).

ఎలా ఆలోచించాలి:

- "Contiguous substring" + "అతి పొడవైన" + "ఒక property (no repeats)" $\rightarrow$ sliding window, **longest valid** రకం.
- Window [**left**, **right**] లో **అన్ని characters unique** అయ్యేలా maintain చేయాలి. కొత్త $s[\text{right}]$ చేర్చినప్పుడు అది ఇప్పటికే window లో ఉంటే — property violate అయింది.
- Violate అయితే ఏం చేయాలి?** ఆ duplicate character window లోంచి పోయేదాకా **left** ని కుడికి కదిపి (shrink) — window మళ్ళీ valid చేయాలి. తర్వాత కొత్త character చేర్చొచ్చు.
- Window లో ఏ characters ఉన్నాయో త్వరగా తెలుసుకోవడానికి ఒక **Set** వాడతాం ($O(1)$ lookup). ప్రతి valid window దగ్గర **right - left + 1** తో max update.
- ఏం గమనించాలి?** ఇది Problem 6 కి అద్దం లాంటిది: అక్కడ **shortest** కోసం valid అయ్యాక shrink చేశాం; ఇక్కడ **longest** కోసం **invalid అయినప్పుడు మాత్రమే** shrink చేసి, లేకపోతే window ని పెంచుతూనే ఉంటాం.

Brute Force: అన్ని substrings (i, j జతలు) generate చేసి, ప్రతిదానికి unique అవునో కాదో Set తో check. Time $O(n^2)$ (లేదా check తో $O(n^3)$), Space $O(n)$. చాలా subarrays పదేపదే re-scan.

Optimal Approach:

- Insight:** ఒక window ని maintain చేసి, duplicate వచ్చినప్పుడు మాత్రం ఎడమ నుండి కుంచిస్తే — ప్రతి character ఎక్కువలో ఎక్కువ ఒకసారి add, ఒకసారి remove $\rightarrow O(n)$.
- Pattern:** Sliding Window — variable, **longest valid**, Set తో.
- Plan:** seen Set, **left=0**, **max=0**. **right** loop: $s[\text{right}]$ ఇప్పటికే seen లో ఉంటే **while** తో **left** ముందుకి (delete చేస్తూ); తర్వాత $s[\text{right}]$ add; **max** update.

Solution (JavaScript):

```
function lengthOfLongestSubstring(s) {  
  const seen = new Set(); // ippati window lo unna characters  
  let left = 0, max = 0;  
  
  for (let right = 0; right < s.length; right++) {  
    // s[right] already window lo unte, aa duplicate poyedaka edama nunchi shrink  
    while (seen.has(s[right])) {  
      seen.delete(s[left]);  
      left++;  
    }  
  }  
}
```

```

    seen.add(s[right]); // ippudu safe ga cherchu
    max = Math.max(max, right - left + 1); // valid window podavu tho update
}
return max;
}

// test
console.log(lengthOfLongestSubstring("abcabcbb")); // 3
console.log(lengthOfLongestSubstring("bbbbb")); // 1
console.log(lengthOfLongestSubstring("pwwkew")); // 3

```

Dry Run: s = "abcabcbb"

right	s[right]	duplicate?	shrink చర్య	seen (window)	max
0	a	✗	—	{a}	1
1	b	✗	—	{a,b}	2
2	c	✗	—	{a,b,c}	3
3	a	✓ (a ఉంది)	delete a, left=1	{b,c,a}	3
4	b	✓	delete b, left=2	{c,a,b}	3
5	c	✓	delete c, left=3	{a,b,c}	3
6	b	✓	delete a → left=4, b ఇంకా ఉంది → delete b, left=5	{c,b}	3
7	b	✓	delete c → left=6, delete b → left=7	{b}	3

Answer **3** ("abc"). Window duplicate వచ్చినప్పుడు మాత్రమే shrink అవుతూ, పొడవైన unique భాగాన్ని పట్టుకుంటోంది.

Complexity:

- **Time: $O(n)$** — **right** n సార్లు; **left** కూడా మొత్తంగా n సార్లు. ప్రతి character Set లోకి ఒకసారి, బయటికి ఒకసారి → amortized $O(n)$.
- **Space: $O(\min(n, \text{alphabet}))$** — Set లో ఎక్కువలో ఎక్కువ window size (unique characters). ASCII అయితే $O(128)$ = $O(1)$.

గుర్తుంచుకోవాల్సినది: "longest substring/subarray with condition" = variable window, **invalid అయినప్పుడు shrink, లేకపోతే expand.** Window content ని త్వరగా track చేయడానికి Set (membership) లేదా Map (counts/positions) వాడతాం. **Optimization:** Map తో ప్రతి character చివరి index store చేస్తే — one-by-one shrink బదులు **left** ని **Math.max(left, lastIndex+1)** కి నేరుగా **jump** చేయొచ్చు (still $O(n)$ కానీ tighter). Interview లో ఈ variant చెప్తే మంచిది.

సాధారణ తప్పులు:

- **while** (seen.has(s[right])) బదులు **if** వాడటం — ఒక్కసారి delete చేసి, duplicate ఇంకా ఉంటే window invalid గా మిగులుతుంది. **while** తప్పనిసరి.

- `left` shrink చేసేటప్పుడు `seen.delete(s[left])` మర్చిపోవడం – Set stale అవుతుంది.
- Substring ని subsequence అనుకోవడం – ఇక్కడ contiguous కావాలి ("pwke" valid కాదు).
- ఖాళీ string కి 0 return మర్చిపోవడం (loop అస్సలు నడవదు, `max=0` సరైనదే – కానీ edge case గుర్తుంచుకో).

8. Substring with Concatenation of All Words (LeetCode #30) – Hard

సమస్య: string `s` మరియు `words` array (అన్ని words ఒకే పొడవు) ఇచ్చారు. `words` లోని ప్రతి word ని సరిగ్గా ఒక్కసారి, ఏ order లోనైనా, మధ్యలో ఖాళీ లేకుండా కలిపితే (concatenation) వచ్చే substrings – వాటి **starting indices** అన్నీ return చెయ్యి. Constraints: $1 \leq s.length \leq 10^4$, $1 \leq words.length \leq 5000$, $1 \leq words[i].length \leq 30$, అన్నీ lowercase.

ఉదాహరణ:

- `s = "barfoothefoobarman"`, `words = ["foo","bar"]` → `[0, 9]`. Index 0: "barfoo" = bar+foo ✓. Index 9: "foobar" = foo+bar ✓.
- `s = "wordgoodgoodgoodbestword"`, `words = ["word","good","best","word"]` → `[]`.
- `s = "barfoofoobarthefoobarman"`, `words = ["bar","foo","the"]` → `[6, 9, 12]`.

ఎలా ఆలోచించాలి:

1. అన్ని words ఒకే పొడవు `L` – ఇది కీలక సరళీకరణ. మొత్తం window పొడవు ఎప్పుడూ `windowLen = L * words.length`. అంటే మనం చూసేది **స్థిర పరిమాణం window** (కానీ word-ల step లో జరుగుతుంది).
2. ఏ order అయినా సరే – అంటే window లోని words **multiset (counts)** `words` counts కి సరిపోవాలి. కాబట్టి `need` = ప్రతి word ఎన్నిసార్లు కావాలో Map.
3. Naive: ప్రతి index `i` దగ్గర, `windowLen` substring ని `L`-పొడవు ముక్కలుగా విడగొట్టి counts match చేయి. ఇది పని చేస్తుంది కానీ ప్రతి index కి పని పునరావృతం.
4. **Insight (optimal):** `s` ని `L`-పొడవు tokens గా చూస్తే – ఒక window ని `L` steps లో slide చేయొచ్చు. కానీ starting alignment `0, 1, ..., L-1` అనే `L` రకాలుగా ఉండొచ్చు. కాబట్టి `L వేర్వేరు offsets` కి, ఒక్కో word-level sliding window నడుపు. ప్రతి offset లో – `right` ని `L` step లో పెంచుతూ word చేర్చు; ఏ word count అవసరం మించితే ఎడమ నుండి words తీసేయి; window నిండితే (`count === numWords`) starting index record చేసి, ఒక word ముందుకి జరుగు.

Brute Force:

```
function findSubstringBrute(s, words) {
  const res = [];
  const L = words[0].length, num = words.length, windowLen = L * num;
  const need = new Map();
  for (const w of words) need.set(w, (need.get(w) || 0) + 1);

  for (let i = 0; i + windowLen <= s.length; i++) {
    const seen = new Map();
    let j = 0;
    for (; j < num; j++) {
      const word = s.substr(i + j * L, L);
```

// j-va muka

```

    if (!need.has(word)) break; // avasaram leni word
    seen.set(word, (seen.get(word) || 0) + 1);
    if (seen.get(word) > need.get(word)) break; // ekkuva sarlu vachhindi
  }
  if (j === num) res.push(i); // anni sarigga serayi
}
return res;
}

```

Time $O(N \times \text{num} \times L)$ ($N = s.length$), Space $O(\text{num})$. ప్రతి start ని independent గా చూస్తుంది.

Optimal Approach:

- **Insight:** L offsets $\times$ word-level sliding window $\rightarrow$ ప్రతి character కిన్ని offsets లో $O(1)$ సార్లు మాత్రమే process అవుతుంది $\rightarrow O(N \times L)$.
- **Pattern:** Sliding Window (fixed word-count window, word-step లో), multiset match తో.
- **Plan:** క్రింది code.

Solution (JavaScript):

```

function findSubstring(s, words) {
  const res = [];
  const L = words[0].length, num = words.length, windowLen = L * num;
  if (s.length < windowLen) return res;

  const need = new Map(); // prati word entha kaavalo
  for (const w of words) need.set(w, (need.get(w) || 0) + 1);

  // L rakala alignments - prati offset ki oka sliding window
  for (let i = 0; i < L; i++) {
    let left = i, count = 0;
    const window = new Map(); // ee window lo unna words

    for (let right = i; right + L <= s.length; right += L) {
      const word = s.substr(right, L); // kudi vaipu kotta word

      if (need.has(word)) {
        window.set(word, (window.get(word) || 0) + 1);
        count++;
        // ee word avasaram minchite, edama nunchi teseyyi
        while (window.get(word) > need.get(word)) {
          const lw = s.substr(left, L);
          window.set(lw, window.get(lw) - 1);
          left += L;
          count--;
        }
        // window nindindi -> starting index record chey, oka word mundhuki
        if (count === num) {
          res.push(left);
          const lw = s.substr(left, L);
          window.set(lw, window.get(lw) - 1);
          left += L;
          count--;
        }
      }
    }
  }
}

```

```

    } else {
        // avasaram leni word vachhindi → window motham reset
        window.clear();
        count = 0;
        left = right + L;
    }
}
}
return res;
}

// test
console.log(findSubstring("barfoothefoobarman", ["foo", "bar"])); // [0, 9]
console.log(findSubstring("barfoofoobarthefoobarman", ["bar","foo","the"])); // [6,9,12]

```

Dry Run: `s = "barfoothefoobarman"` , `words = ["foo","bar"]` , `L=3` , `num=2` , `need={foo:1, bar:1}` .
Offset `i=0` :

right	word	need?	చర్య	window	count	left
0	bar	✓	add	{bar:1}	1	0
3	foo	✓	add → count=2 → push 0 ; remove bar; left=3	{bar:0,foo:1}	1	3
6	the	✗	reset; left=9	{}	0	9
9	foo	✓	add	{foo:1}	1	9
12	bar	✓	add → count=2 → push 9 ; remove foo; left=12	{foo:0,bar:1}	1	12
15	man	✗	reset; left=18	{}	0	18

Offset 0 result: `[0, 9]` . Offsets 1, 2 లో valid window రాదు. చివరి answer `[0, 9]` .

Complexity:

- Time: $O(N \times L)$** — $N = s.length$. L offsets, ఒక్కో offset $O(N/L)$ word-steps, ప్రతి step లో substr $O(L) \rightarrow O(L \times (N/L) \times L) = O(N \times L)$. Brute $(O(N \cdot \text{num} \cdot L))$ కంటే మెరుగు (సాధారణంగా $\text{num} \gg L$).
- Space: $O(\text{num} \times L)$** — need/window maps లో words.

గుర్తుంచుకోవాల్సినది: "ఒకే పొడవు tokens", "ప్రతిదే ఒకసారి / permutation", "concatenation" → **word-level fixed sliding window with L offsets + multiset (Map) match**. ఇది "Find All Anagrams" (character-level) యొక్క word-level version. కీలక ఉపాయం: characters బదులు **fixed-length blocks** ని units గా చూడటం, మరియు $0 \dots L-1$ alignments అన్నీ cover చేయడం.

సాధారణ తప్పులు:

- L offsets మర్చిపోవడం** — ఒక్క offset 0 తో నడిపితే మధ్యలో మొదలయ్యే valid windows miss (ఇది #1 తప్పు).
- Multiset (counts) బదులు Set వాడటం — duplicate words (`["word","word"]`) ఉంటే తప్పు.
- Window నిండాక `left` ని ముందుకీ జరపడం మర్చిపోవడం → అదే index పదేపదే / infinite.
- `s.length < windowLen` edge case check మర్చిపోవడం.

9. Minimum Window Substring (LeetCode #76) — Hard

సమస్య: రెండు strings s మరియు t ఇచ్చారు. t లోని ప్రతి character ని (duplicates తో సహా, అవసరమైన సంఖ్యలో) కలిగిన s యొక్క అతి చిన్న (minimum length) window (substring) ని return చెయ్యి. అలాంటిది లేకపోతే "" . సమాధానం unique అని హామీ. Constraints: $1 \leq s.length, t.length \leq 10^5$, English letters (upper + lower).

ఉదాహరణ:

- $s = \text{"ADOBECODEBANC"} , t = \text{"ABC"} \rightarrow \text{"BANC"}$ (A, B, C మూడూ ఉన్న అతి చిన్న భాగం).
- $s = \text{"a"} , t = \text{"a"} \rightarrow \text{"a"} .$
- $s = \text{"a"} , t = \text{"aa"} \rightarrow \text{""} (s లో a ఒక్కటే, t కి రెండు కావాలి).$

ఎలా ఆలోచించాలి:

- "Contiguous window" + "అతి చిన్న" + "ఒక condition ని satisfy" $\rightarrow$ sliding window, **shortest valid** (Problem 6 లాంటిదే, కానీ condition complex).
- Condition: window లో t లోని ప్రతి character కావలసినంత ఉండాలి. కాబట్టి $need = t$ యొక్క character counts.
- సవాలు: window valid అని $O(1)$ లో ఎలా తెలుసుకోవాలి?** ప్రతిసారి రెండు maps పోల్చడం ఖరీదు. ఉపాయం — $required = t$ లోని **unique** characters సంఖ్య; $formed =$ ఇప్పటిదాకా window లో కావలసినంత సంఖ్యకి చేరిన unique characters. $formed == required$ అయితే window valid. ఇది $O(1)$ check!
- Plan:** **right** పెంచుతూ character చేర్చు; ఏ character దాని $need$ count కి సరిగ్గా చేరితే $formed++$. $formed == required$ (valid) అయినంతవరకు — best update చేసి, ఎడమ నుండి shrink చేయి; shrink వల్ల ఏదైనా character count $need$ కంటే తగ్గితే $formed--$.
- ఏం గమనించాలి?** $formed$ counter వల్ల "window valid అవునా?" అనేది constant time — ఇదే ఈ solution ని $O(n)$ గా ఉంచుతుంది.

Brute Force: అన్ని substrings (i, j) generate చేసి, ప్రతిదానికి t counts ఉన్నాయో check. Time $O(n^2 \times |t|)$ (లేదా $O(n^3)$), Space $O(|t|)$. చాలా నెమ్మది.

Optimal Approach:

- Insight:** $need / window$ counts + $formed / required$ counters తో $O(1)$ validity check $\rightarrow$ variable window $O(n)$.
- Pattern:** Sliding Window — variable, **shortest valid**, count-matching తో.
- Plan:** క్రింది code.

Solution (JavaScript):

```
function minWindow(s, t) {
    if (s.length < t.length) return "";

    const need = new Map(); // t lo prati character entha kaavalo
    for (const c of t) need.set(c, (need.get(c) || 0) + 1);
    const required = need.size; // entha UNIQUE chars satisfy avvali
```

```

const window = new Map();           // ippati window lo counts
let formed = 0;                     // entha unique chars kaavalsinantha unnayi
let left = 0;
let best = [Infinity, 0, 0];        // [podavu, left, right]

for (let right = 0; right < s.length; right++) {
  const c = s[right];
  window.set(c, (window.get(c) || 0) + 1);    // EXPAND

  // ee character sarigga need count ki cherite, oka requirement teerindi
  if (need.has(c) && window.get(c) === need.get(c)) {
    formed++;
  }

  // window valid (formed == required) ainanta varaku SHRINK
  while (formed === required) {
    if (right - left + 1 < best[0]) {          // chinna window dorikite record
      best = [right - left + 1, left, right];
    }
    const lc = s[left];                       // edama character teseyyi
    window.set(lc, window.get(lc) - 1);
    if (need.has(lc) && window.get(lc) < need.get(lc)) {
      formed--;                               // ippudu ee requirement teerteledu
    }
    left++;
  }
}

return best[0] === Infinity ? "" : s.substring(best[1], best[2] + 1);
}

// test
console.log(minWindow("ADOBECODEBANC", "ABC")); // "BANC"
console.log(minWindow("a", "a"));               // "a"
console.log(minWindow("a", "aa"));              // ""

```

Dry Run: s = "ADOBECODEBANC" , t = "ABC" → need={A:1,B:1,C:1} , required=3 (కోలక ఘట్టాలు)

right	char	formed	window valid?	shrink → best
5	C	3	✓	window [0..5]="ADOBEC" (len 6); shrink A తీస్తే formed=2 → best=[6,0,5]
10	A	3	✓	[1..10] shrink → D,O,B,E,C తీసి len 6 దగ్గర ఆగదు, C తీస్తే formed=2; best ఇంకా 6
12	C	3	✓	[6..12] shrink → O,D తీసి len 5 ("EBANC") → best=[5,8,12], ఇంకా E తీసి len 4 ("BANC") → best=[4,9,12] , B తీస్తే formed=2

చివరి best = [4, 9, 12] → s.substring(9, 13) = "BANC". Window t ని కవర్ చేసినంతవరకు కుంచించి, అన్ని valid windows లో అతి చిన్నది పట్టుకుంది.

Complexity:

- **Time:** $O(|s| + |t|)$ — t ని ఒకసారి scan (need build), s ని sliding window లో ఒకసారి (**right** , **left** కలిపి $2 \cdot |s|$ కదలికలు). formed వల్ల validity check $O(1)$.
- **Space:** $O(|s| + |t|)$ — worst case maps లో అన్ని unique characters. Alphabet fixed అయితే $O(1)$.

గుర్తుంచుకోవాల్సినది: "అన్ని required characters/elements ని కలిగిన **అతి చిన్న** window" = variable sliding window + **need / window counts** + **formed / required counters**. ఈ formed trick — "ఇప్పటిదాకా ఎన్ని requirements తీరాయి" ని $O(1)$ లో track చేయడం — చాలా hard window problems (అన్ని anagram/permutation/cover problems) కి master technique. Shortest కాబట్టి valid అయ్యాక aggressive shrink.

సాధారణ తప్పులు:

- `formed++` ని `window.get(c) === need.get(c)` (సరిగ్గా సమానం) దగ్గర చేయాలి — $\geq$ వాడితే duplicates కి పలుసార్లు లెక్కించి తప్పు.
- Shrink లో `formed--` ని `window.get(lc) < need.get(lc)` (తక్కువైనప్పుడు మాత్రమే) చేయాలి — లేకపోతే extra copies తీసినప్పుడూ తప్పుగా తగ్గిస్తుంది.
- Best window ని sum/length కాకుండా actual substring return చేయాలి — indices store చేసి చివర్లో slice.
- `required = need.size` (unique count) — `t.length` కాదు (t లో duplicates ఉంటే తేడా వస్తుంది).

Pattern: Matrix Traversal

వివరణ

Matrix అంటే rows $\times$ columns గల 2D grid (`matrix[row][col]`). m = rows, n = columns. Matrix problems లో pointers ఉండవు కానీ **boundaries, directions, in-place encoding** అనే మూడు ఆలోచనలు మళ్ళీ మళ్ళీ వస్తాయి. వీటిని master చేస్తే matrix problems అన్నీ ఒకేలా కనిపిస్తాయి.

1. Boundaries (సరిహద్దులు): **top** , **bottom** , **left** , **right** అనే నాలుగు గోడలు పెట్టి, వాటిని లోపలికి కుంచిస్తూ matrix ని పొరలుగా (layers) traverse చేయడం. **Spiral** traversal, layer-by-layer processing కి ఇదే గుండె.

2. Direction vectors (దిశలు): కదలికల్ని `[dr, dc]` జతలుగా చూడటం. ఉదా. కుడి = `[0, 1]` , కింద = `[1, 0]` , ఎడమ = `[0, -1]` , పైకి = `[-1, 0]` . కొత్త స్థానం = `[r+dr, c+dc]` . **if/else** ల ప్రవాహం బదులు ఈ array తో loop వేసి neighbors ని శుభ్రంగా చూడొచ్చు (Game of Life, grid DFS/BFS).

3. In-place tricks (అదనపు memory లేకుండా): Matrix problems లో తరచూ " **$O(1)$ extra space** లో చెయ్యి" అంటారు. అప్పుడు కొత్త matrix కాకుండా — ఉన్న matrix నే **encoding** గా వాడతాం. మూడు ప్రధాన ఉపాయాలు:

- **First row/column ని markers** గా వాడటం (Set Matrix Zeroes).
- **Sign / extra state ని ఒకే cell లో encode** చేయటం (Game of Life లో $0 \rightarrow 1$ మార్పుని ప్రత్యేక codes తో).
- **Transpose + reverse** (matrix ని rotate చేయటానికి $O(1)$ space).

Real-life Scenario

Spiral / boundaries = ఉల్లిపాయ ఒలవడం. ఉల్లిపాయ బయటి పొరని (top row → right col → bottom row → left col) ఒలిచి పక్కన పెడతావు, తర్వాత లోపలి పొర. నాలుగు గోడలు (top/bottom/left/right) ప్రతి పొర తర్వాత ఒక అడుగు లోపలికి జరుగుతాయి – చివరికి మధ్య cell దగ్గర ఆగుతావు.

In-place encoding = పుస్తకం margin లో గుర్తు. కొత్త notebook (extra matrix) తీసుకోకుండా, ఇప్పటికే ఉన్న పేజీ margin లోనే చిన్న గుర్తు రాసుకుంటావు – తర్వాత అవసరం తీరాక ఆ గుర్తుని చెరిపేస్తావు. Cell లోని original విలువని పాడుచేయకుండా అదనపు సమాచారం దాచడం.

ఎప్పుడు వాడాలి / ఎలా గుర్తించాలి

- "matrix", "grid", "2D array", "m x n board" – స్పష్టంగా matrix problem.
- "spiral", "rotate 90°", "transpose", "diagonal" → boundaries / coordinate-mapping tricks.
- "in-place", "O(1) space" on a grid → **encoding** ఆలోచించు (first row/col markers, sign, transpose+reverse).
- "neighbors", "adjacent cells", "8 directions" → direction vectors array.
- Row/col/box లాంటి **groups లో constraint check** → cell → group-index mapping ($\text{Math.floor}(r/3)*3 + \text{Math.floor}(c/3)$ లాంటివి).

Reusable Templates (JavaScript)

```
// (a) Direction vectors – 4 disaila neighbors
const dirs = [[0, 1], [1, 0], [0, -1], [-1, 0]]; // kudi, kinda, edama, paiki
function neighbors(r, c, m, n) {
  const out = [];
  for (const [dr, dc] of dirs) {
    const nr = r + dr, nc = c + dc;
    if (nr >= 0 && nr < m && nc >= 0 && nc < n) out.push([nr, nc]); // haddullo unte
  }
  return out;
}

// (b) Boundary-based spiral peel
function spiralPeel(matrix, visit) {
  let top = 0, bottom = matrix.length - 1;
  let left = 0, right = matrix[0].length - 1;
  while (top <= bottom && left <= right) {
    for (let c = left; c <= right; c++) visit(top, c); // paina: edama→kudi
    top++;
    for (let r = top; r <= bottom; r++) visit(r, right); // kudi: paina→kinda
    right--;
    if (top <= bottom)
      for (let c = right; c >= left; c--) visit(bottom, c); // kinda: kudi→edama
    bottom--;
    if (left <= right)
      for (let r = bottom; r >= top; r--) visit(r, left); // edama: kinda→paina
    left++;
  }
}
```

```
// (c) In-place 90° rotation = transpose + prati row reverse
function rotate90(m) {
  const n = m.length;
  for (let i = 0; i < n; i++) // transpose: m[i][j] <-> m[j][i]
    for (let j = i + 1; j < n; j++)
      [m[i][j], m[j][i]] = [m[j][i], m[i][j]];
  for (const row of m) row.reverse(); // prati row ni reverse
}
```

Complexity

- **Time: $O(m \times n)$** – దాదాపు అన్ని matrix traversals ప్రతి cell ని constant సార్లు (1 లేదా 2) చూస్తాయి.
- **Space: $O(1)$** in-place encoding తో; output తయారు చేస్తే $O(m \times n)$ (అది అనివార్యం).

10. Valid Sudoku (LeetCode #36) – Medium

సమస్య: 9×9 Sudoku board ని validate చెయ్యి. **నింపిన cells** మాత్రమే check చేయాలి, ఈ rules ప్రకారం: (1) ప్రతి **row** లో 1–9 digits పునరావృతం కాకూడదు; (2) ప్రతి **column** లో పునరావృతం కాకూడదు; (3) ప్రతి 3×3 **sub-box** లో పునరావృతం కాకూడదు. ఖాళీ cells `'.'`. Board solvable కావాలని లేదు – ప్రస్తుతం ఉన్నవి rules ఉల్లంఘించకపోతే చాలు. Constraints: board 9×9 , cells digits `'1'–'9'` లేదా `'.'`.

ఉదాహరణ: valid board (కొన్ని cells నింపినవి) → `true`. ఒకే row/col/box లో ఒకే digit రెండుసార్లు ఉంటే → `false`.

ఎలా ఆలోచించాలి:

1. మూడు rules – row, column, box. ప్రతి digit ని దాని **row, column, box మూడింటిలోనూ** ఇప్పటికే చూశామా అని check చేయాలి.
2. "ఇప్పటికే చూశామా?" = **Set membership** ($O(1)$). 9 rows కి 9 Sets, 9 cols కి 9 Sets, 9 boxes కి 9 Sets.
3. **కీలక ఉపాయం – box index ఎలా?** (`r`, `c`) ఏ 3×3 box లో ఉందో ఒకే సూత్రంతో: `box = Math.floor(r/3) * 3 + Math.floor(c/3)`. (`r/3` ఏ అడ్డు band – 0/1/2; `*3 + c/3` ఏ నిలువు band – కలిపి 0..8.)
4. **ఏం గమనించాలి?** ఒకే pass లో మూడు constraints చూడొచ్చు – ప్రతి cell దగ్గర row/col/box Sets లో digit ఉందేమో చూసి, లేకపోతే మూడింటికీ add.

Brute Force: ప్రతి row విడిగా, ప్రతి col విడిగా, ప్రతి box విడిగా – మూడు separate passes, ప్రతిదానిలో duplicates ఉన్నాయో చూడటం. పని చేస్తుంది కానీ board ని 3 సార్లు తిరుగుతాం, code పొడవు.

Optimal Approach:

- **Insight:** 27 Sets (9 rows + 9 cols + 9 boxes) తో ఒకే traversal లో మూడు constraints. Box-index mapping సూత్రం core.
- **Pattern:** Matrix traversal + hashing (group-index mapping).
- **Plan:** క్రింది code – single pass.

Solution (JavaScript):

```
function isValidSudoku(board) {
  // 9 rows, 9 cols, 9 boxes ki ververu Sets
  const rows = Array.from({ length: 9 }, () => new Set());
  const cols = Array.from({ length: 9 }, () => new Set());
  const boxes = Array.from({ length: 9 }, () => new Set());

  for (let r = 0; r < 9; r++) {
    for (let c = 0; c < 9; c++) {
      const val = board[r][c];
      if (val === ".") continue; // khaali cell - skip

      const b = Math.floor(r / 3) * 3 + Math.floor(c / 3); // box index (0..8)

      // ee digit row/col/box lo already unda? unte invalid
      if (rows[r].has(val) || cols[c].has(val) || boxes[b].has(val)) {
        return false;
      }
      rows[r].add(val); // mooduintiki add
      cols[c].add(val);
      boxes[b].add(val);
    }
  }
  return true; // eppudu clash raaledu → valid
}
```

Dry Run: board లో board[0][0]='5', board[0][1]='3', తర్వాత board[1][0]='6' ...

- (0,0)='5' : box = $0*3+0 = 0$. rows[0], cols[0], boxes[0] లో 5 లేదు → మూడింటికీ add.
- (0,1)='3' : box = 0. rows[0] లో ఇప్పుడు {5}, 3 లేదు → add. rows[0]= {5,3}.
- ...ఇలా. ఒకవేళ మళ్ళీ row 0 లో ఇంకో cell 5 వస్తే → rows[0].has('5') true → **return false**.
- box index తనిఖీ: (4,4) → $\text{Math.floor}(4/3)*3 + \text{Math.floor}(4/3) = 1*3+1 = 4$ (మధ్య box). (8,8) → $2*3+2 = 8$ (కుడి-కింద box). సూత్రం సరిగ్గా పని చేస్తోంది.

Complexity:

- **Time: O(1)** – board ఎప్పుడూ $9 \times 9 = 81$ cells (fixed). సాంకేతికంగా $O(m \times n)$ కానీ ఇక్కడ constant.
- **Space: O(1)** – 27 Sets, ఒక్కోదాంట్లో ≤ 9 elements – constant.

గుర్తుంచుకోవాల్సినది: "Grid లో row/column/box లాంటి **groups లో uniqueness/constraint check**" → ప్రతి group కి ఒక Set + cell → **group-index mapping సూత్రం**. ఇక్కడ box mapping $\text{Math.floor}(r/3)*3 + \text{Math.floor}(c/3)$ – ఈ "సబ్-గ్రిడ్ index" ఉపాయం చాలా grid problems లో వస్తుంది. ఒకే pass లో బహుళ constraints చూడటం కూడా గుర్తుంచుకోదగ్గ pattern.

సాధారణ తప్పులు:

- **Box index సూత్రం తప్పు** – $r/3 + c/3$ ($\times 3$ మర్చిపోవడం) → boxes కలిసిపోతాయి. $\text{Math.floor}(r/3)*3 + \text{Math.floor}(c/3)$ సరైనది.
- '.' cells ని skip చేయకపోవడం → ఖాళీలని duplicate అనుకుంటుంది.
- Value ని number గా parse చేయడం అనవసరం – string '5' నే Set లో పెట్టొచ్చు.
- Board solvable అవునా అని check చేయడం – అది ఈ problem కాదు, ప్రస్తుత validity మాత్రమే.

11. Spiral Matrix (LeetCode #54) – Medium

సమస్య: $m \times n$ matrix ఇచ్చారు. దానిలోని అన్ని elements ని **spiral order** (బయటి పొర నుండి లోపలికి, గడియారం దిశలో) లో ఒక array గా return చెయ్యి. Constraints: $1 \leq m, n \leq 10$, $-100 \leq \text{matrix}[i][j] \leq 100$.

ఉదాహరణ:

- $[[1,2,3],[4,5,6],[7,8,9]] \rightarrow [1,2,3,6,9,8,7,4,5]$.
- $[[1,2,3,4],[5,6,7,8],[9,10,11,12]] \rightarrow [1,2,3,4,8,12,11,10,9,5,6,7]$.

ఎలా ఆలోచించాలి:

- Spiral అంటే – పైన ఎడమ నుండి కుడికి → కుడి పక్క పైనుంచి కిందకి → కింద కుడి నుండి ఎడమకి → ఎడమ పక్క కింద నుంచి పైకి → తర్వాత ఒక పొర లోపలికి. ఇది **ఉల్లిపాయ ఒలిచినట్టు**, పొరలు పొరలుగా.
- ప్రతి పొరని నాలుగు గోడలతో సూచించొచ్చు: **top**, **bottom**, **left**, **right**. ఒక పొర పూర్తయ్యాక ఆ గోడ ఒక అడుగు లోపలికి జరుగుతుంది ($top++$, $right--$, $bottom--$, $left++$).
- Plan:** while ($top \leq bottom$ & $left \leq right$) – నాలుగు అంచుల్ని వరుసగా traverse: (a) top row $L \rightarrow R$, $top++$; (b) right col $T \rightarrow B$, $right--$; (c) bottom row $R \rightarrow L$, $bottom--$; (d) left col $B \rightarrow T$, $left++$.
- ఏం గమనించాలి – అతి ముఖ్యమైన మెలిక:** matrix square కాకపోతే, చివర్లో ఒక row లేదా ఒక column మిగలొచ్చు. అప్పుడు bottom row / left col ని **మళ్ళీ లెక్కించే (double count)** ప్రమాదం. అందుకే (c) కి ముందు $if (top < bottom)$, (d) కి ముందు $if (left < right)$ – ఈ guards తప్పనిసరి.

Brute Force: ఇక్కడ "brute force" అంటూ ప్రత్యేకంగా ఏమీ లేదు – direction ని if/else తో మార్చుకుంటూ, ప్రతి visited cell ని ఒక visited matrix లో గుర్తు పెట్టి, గోడకో/visited cell కో తగిలితే తిరగడం. ఇది $O(m \cdot n)$ time కానీ **$O(m \cdot n)$ extra space** (visited) + code గజిబిజి. Boundary approach దీన్ని $O(1)$ space లో, శుభ్రంగా చేస్తుంది.

Optimal Approach:

- Insight:** గోడల్ని (boundaries) explicit గా track చేస్తే – visited matrix అవసరం లేదు, direction logic సూటిగా ఉంటుంది.
- Pattern:** Matrix traversal – boundary-based (layer peeling).
- Plan:** పైన చెప్పిన నాలుగు అంచుల loop + రెండు guards.

Solution (JavaScript):

```
function spiralOrder(matrix) {
    const res = [];
    if (matrix.length === 0) return res;

    let top = 0, bottom = matrix.length - 1;
    let left = 0, right = matrix[0].length - 1;

    while (top <= bottom && left <= right) {
        // (a) paina row: edama → kudi
        for (let c = left; c <= right; c++) res.push(matrix[top][c]);
        top++;

        // (b) kudi column: paina → kinda
```

```

for (let r = top; r <= bottom; r++) res.push(matrix[r][right]);
right--;

// (c) kinda row: kudi → edama (row inka migilite matrame)
if (top <= bottom) {
  for (let c = right; c >= left; c--) res.push(matrix[bottom][c]);
  bottom--;
}

// (d) edama column: kinda → paina (column inka migilite matrame)
if (left <= right) {
  for (let r = bottom; r >= top; r--) res.push(matrix[r][left]);
  left++;
}
}
return res;
}

// test
console.log(spiralOrder([[1,2,3],[4,5,6],[7,8,9]])); // [1,2,3,6,9,8,7,4,5]
console.log(spiralOrder([[1,2,3,4],[5,6,7,8],[9,10,11,12]])); // [1,2,3,4,8,12,11,10,9,5,6,7]

```

Dry Run: `[[1,2,3],[4,5,6],[7,8,9]]` (`top=0,bottom=2,left=0,right=2`)

పాఠ	చర్య	తీసుకున్నవి	గోడల update
1	top row L→R	1, 2, 3	top=1
1	right col T→B	6, 9	right=1
1	bottom row R→L	8, 7	bottom=1
1	left col B→T	4	left=1
2	top row (row 1, c=1)	5	top=2
2	right col (r=2..1 ఖాళీ)	—	right=0
2	top(2)>bottom(1) → skip	—	—
—	top>bottom → loop ముగింపు		

Result: `[1,2,3,6,9,8,7,4,5]` . గోడలు లోపలికి జరుగుతూ మధ్య cell 5 దగ్గర ఆగాయి.

Complexity:

- **Time: $O(m \times n)$** — ప్రతి cell సరిగ్గా ఒక్కసారి visit అవుతుంది.
- **Space: $O(1)$** — output తప్ప (అది అనివార్యం) అదనపు memory లేదు; visited matrix అవసరం లేదు.

గుర్తుంచుకోవాల్సినది: "Spiral", "layer by layer", "perimeter then inward" → **four boundaries**

(**top/bottom/left/right**) ని కుంచిస్తూ **traverse**. ఈ boundary-peeling నమూనా Spiral Matrix II (నింపడం), matrix ని పొరలుగా process చేసే అన్ని problems కి. అతి కీలకం: single-row/single-column మిగిలినప్పుడు double-count ఆపే **రెండు if guards**.

సాధారణ తప్పులు:

- **if (top <= bottom) / if (left <= right) guards మర్చిపోవడం** – non-square matrix లో చివరి strip elements రెండుసార్లు వస్తాయి (ఈ problem లో #1 తప్పు).
- Loop condition ఓ బదులు `||` రాయడం – ఒక dimension అయిపోయాక కూడా కొనసాగి out-of-bounds.
- గోడల్ని update చేయకపోవడం (`top++` మొ.) → infinite loop.
- ఖాళీ matrix / ఒకే row / ఒకే column edge cases test చేయకపోవడం.

12. Rotate Image (LeetCode #48) – Medium

సమస్య: $n \times n$ matrix ని 90° గడియారం దిశలో (clockwise) తిప్పు. In-place గా చేయాలి – కొత్త matrix కేటాయించకూడదు ($O(1)$ extra space). Constraints: $1 \leq n \leq 20$, $-1000 \leq \text{matrix}[i][j] \leq 1000$.

ఉదాహరణ:

- `[[1,2,3],[4,5,6],[7,8,9]] → [[7,4,1],[8,5,2],[9,6,3]]`.
- మొదటి row `[1,2,3]` తిప్పితే చివరి column `[1,2,3]` (ఫైనలించి కిందకి) అవుతుంది.

ఎలా ఆలోచించాలి:

1. 90° clockwise లో ఒక cell ఎక్కడికి వెళ్తుంది? Coordinate mapping చూద్దాం: $(r, c) \rightarrow (c, n-1-r)$. ఉదా. $(0,0)$ (విలువ 1) $\rightarrow (0, 2)$ (కుడి పైమూల). నిజంగా output లో 1 కుడి పైన ఉంది. ✓
2. In-place గా ఈ mapping ని నేరుగా చేయడం గజిబిజి (cycles లో 4 cells కదులుతాయి). రెండు simple steps గా విడగొడితే చాలా సులభం:
 - **Step 1 – Transpose:** $\text{matrix}[i][j] \leftrightarrow \text{matrix}[j][i]$ swap (diagonal మీదుగా అడ్డం). ఇది $(r, c) \rightarrow (c, r)$.
 - **Step 2 – ప్రతి row ని reverse:** $(c, r) \rightarrow (c, n-1-r)$.
 - రెండూ కలిపి: $(r, c) \rightarrow (c, n-1-r)$ = సరిగ్గా 90° clockwise! 🌟
3. **ఏం గమనించాలి?** Transpose చేసేటప్పుడు j ని $i+1$ నుండి మొదలుపెట్టాలి – లేకపోతే ప్రతి జతని రెండుసార్లు swap చేసి తిరిగి మొదటికి వస్తుంది (diagonal కి ఇరువైపులా).

Brute Force: కొత్త $n \times n$ matrix `result` తీసుకుని `result[c][n-1-r] = matrix[r][c]` పెట్టడం. Time $O(n^2)$, కానీ **Space $O(n^2)$** – in-place constraint ఉల్లంఘన. అర్థం చేసుకోవడానికి బాగుంది కానీ ఈ problem కి సరిపోదు.

Optimal Approach:

- **Insight:** 90° clockwise = **Transpose + row reverse**. రెండూ in-place, $O(1)$ extra space.
- **Pattern:** Matrix traversal – in-place transform (coordinate mapping).
- **Plan:** క్రింది code – రెండు passes.

Solution (JavaScript):

```
function rotate(matrix) {  
    const n = matrix.length;
```

```
// Step 1: Transpose – matrix[i][j] <=> matrix[j][i] (j > i matrame)
for (let i = 0; i < n; i++) {
  for (let j = i + 1; j < n; j++) {
    [matrix[i][j], matrix[j][i]] = [matrix[j][i], matrix[i][j]]; // swap
  }
}

// Step 2: prati row ni reverse
for (let i = 0; i < n; i++) {
  matrix[i].reverse();
}

// test
const m = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];
rotate(m);
console.log(m); // [[7,4,1],[8,5,2],[9,6,3]]
```

Dry Run: $[[1,2,3], [4,5,6], [7,8,9]]$, $n=3$

Step 1 — Transpose (diagonal మీదుగా అడ్డం):

- $(0,1) \leftrightarrow (1,0) : 2 \leftrightarrow 4$. $(0,2) \leftrightarrow (2,0) : 3 \leftrightarrow 7$. $(1,2) \leftrightarrow (2,1) : 6 \leftrightarrow 8$.
- ఫలితం: $[[1,4,7], [2,5,8], [3,6,9]]$.

Step 2 — ప్రతి row reverse:

- $[1,4,7] \rightarrow [7,4,1]$
- $[2,5,8] \rightarrow [8,5,2]$
- $[3,6,9] \rightarrow [9,6,3]$
- ఫలితం: $[[7,4,1], [8,5,2], [9,6,3]]$. సరిగ్గా 90° clockwise!

Complexity:

- **Time: $O(n^2)$** — transpose $\sim n^2/2$ swaps, reverse n rows $\times n/2$ — కలిపి $O(n^2)$. ప్రతి element constant సార్లు తాకుతుంది.
- **Space: $O(1)$** — swaps, reverse రెండూ in-place. కొత్త matrix లేదు.

గుర్తుంచుకోవాల్సినది: Matrix rotation = **coordinate mapping** ని రెండు **simple in-place ops** గా విడగొట్టడం.

- **90° clockwise** = transpose + ప్రతి **row** reverse.
- **90° counter-clockwise** = transpose + ప్రతి **column** reverse (లేదా ముందు row reverse, తర్వాత transpose).
- **180°** = ప్రతి row reverse + ప్రతి column reverse (లేదా రెండుసార్లు 90°). ఈ decomposition గుర్తుంటే ఏ rotation అయినా వెంటనే రాయగలవు. Alternative: బయటి పొర నుండి 4-cell cyclic swaps (కష్టం కానీ single pass).

సాధారణ తప్పులు:

- Transpose లో j ని 0 నుండి మొదలుపెట్టడం $\rightarrow$ ప్రతి swap రెండుసార్లు జరిగి matrix మారదు. $j = i+1$ తప్పనిసరి.
- Clockwise కి row reverse, counter-clockwise కి column reverse — తారుమారు చేస్తే తప్పు దిశ.

- కొత్త matrix return చేయడం – problem **in-place** modify అడుగుతోంది (return void).
- Non-square matrix కి ఈ transpose వాడటం – ఇది $n \times n$ కి మాత్రమే (rectangular కి dimensions మారతాయి).

13. Set Matrix Zeroes (LeetCode #73) – Medium

సమస్య: $m \times n$ matrix ఇచ్చారు. ఏదైనా cell 0 అయితే – ఆ మొత్తం row మరియు మొత్తం column ని 0 చెయ్యి. **In-place** గా చేయాలి. Ideal గా **O(1) extra space**. Constraints: $1 \leq m, n \leq 200$, $-2^{31} \leq \text{matrix}[i][j] \leq 2^{31}-1$.

ఉదాహరణ:

- $\begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 1 \end{bmatrix}$.
- $\begin{bmatrix} 0 & 1 & 2 & 0 \\ 3 & 4 & 5 & 2 \\ 1 & 3 & 1 & 5 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 4 & 5 & 0 \\ 0 & 3 & 1 & 0 \end{bmatrix}$.

ఎలా ఆలోచించాలి:

1. **అతిపెద్ద ఉచ్చు (trap):** cell 0 కనిపించిన వెంటనే దాని row/col ని అక్కడికక్కడే 0 చేస్తే – ఆ కొత్త 0 లు తర్వాత ఇంకొక row/col ని తప్పుగా 0 చేయిస్తాయి (cascade). కాబట్టి **ముందు ఏ rows/cols ని 0 చేయాలో గుర్తుంచుకుని, తర్వాత ఒకసారి** అమలు చేయాలి.
2. Naive గా – ఏ rows, cols లో 0 ఉందో రెండు Sets/arrays లో store చేయొచ్చు. అది **O(m+n) space**. పర్యాలేదు కానీ constraint "O(1)" అంటే?
3. **O(1) insight – in-place markers:** అదనపు arrays బదులు, **మొదటి row & మొదటి column నే markers** గా వాడు! $\text{matrix}[r][c] == 0$ అయితే – $\text{matrix}[r][0] = 0$ (ఈ row కి గుర్తు) మరియు $\text{matrix}[0][c] = 0$ (ఈ col కి గుర్తు) పెట్టు.
4. **మెలిక:** మొదటి row/col నే markers గా వాడుతున్నాం కాబట్టి, వాటికి వాటికి 0 కావాలో లేదో వేరుగా తెలుసుకోవాలి – **రెండు boolean flags** (firstRowZero, firstColZero) తో ముందే record చేసుకో.
5. **Plan (order ముఖ్యం):** (1) flags set చెయ్యి; (2) inner cells ($r \geq 1, c \geq 1$) scan చేసి markers పెట్టు; (3) markers చూసి inner cells ని 0 చెయ్యి; (4) చివర్లో flags బట్టి మొదటి row/col ని 0 చెయ్యి.

Brute Force / O(m+n): 0 ఉన్న rows, cols ని రెండు Sets లో గుర్తించి, తర్వాత ఆ rows/cols అన్నీ 0 చెయ్యి. Time **O(m·n)**, Space **O(m+n)**. సరిగ్గా పని చేస్తుంది, కానీ O(1) కాదు.

Optimal Approach:

- **Insight:** మొదటి row/col ని marker storage గా reuse చేస్తే – extra space O(1). రెండు flags తో first row/col ని ప్రత్యేకంగా handle.
- **Pattern:** Matrix traversal – in-place encoding (markers).
- **Plan:** పైన 4 steps.

Solution (JavaScript):

```
function setZeroes(matrix) {
    const m = matrix.length, n = matrix[0].length;
    let firstRowZero = false, firstColZero = false;

    // Step 1: modati row/col lo 0 unda? (viduga flag)
```

```

for (let c = 0; c < n; c++) if (matrix[0][c] === 0) firstRowZero = true;
for (let r = 0; r < m; r++) if (matrix[r][0] === 0) firstColZero = true;

// Step 2: inner cells lo 0 unte, aa row/col markers ni (first row/col lo) pettu
for (let r = 1; r < m; r++) {
  for (let c = 1; c < n; c++) {
    if (matrix[r][c] === 0) {
      matrix[r][0] = 0; // ee row ki gurtu
      matrix[0][c] = 0; // ee col ki gurtu
    }
  }
}

// Step 3: markers chusi inner cells ni 0 chey
for (let r = 1; r < m; r++) {
  for (let c = 1; c < n; c++) {
    if (matrix[r][0] === 0 || matrix[0][c] === 0) matrix[r][c] = 0;
  }
}

// Step 4: chivarlo flags batti modati row/col ni 0 chey
if (firstRowZero) for (let c = 0; c < n; c++) matrix[0][c] = 0;
if (firstColZero) for (let r = 0; r < m; r++) matrix[r][0] = 0;
}

// test
const a = [[1, 1, 1], [1, 0, 1], [1, 1, 1]];
setZeroes(a);
console.log(a); // [[1,0,1],[0,0,0],[1,0,1]]

```

Dry Run: $[[1,1,1], [1,0,1], [1,1,1]]$ ($m=3, n=3$)

- **Step 1:** row 0 = $[1,1,1] \rightarrow \text{firstRowZero}=\text{false}$. col 0 = $1,1,1 \rightarrow \text{firstColZero}=\text{false}$.
- **Step 2:** inner scan లో $\text{matrix}[1][1]=0 \rightarrow$ markers: $\text{matrix}[1][0]=0$, $\text{matrix}[0][1]=0$. ఇప్పుడు matrix = $[[1,0,1], [0,0,1], [1,1,1]]$.
- **Step 3:** markers చూసి — row 1 (marker $\text{matrix}[1][0]=0$) $\rightarrow \text{matrix}[1][2]=0$. col 1 (marker $\text{matrix}[0][1]=0$) $\rightarrow \text{matrix}[2][1]=0$. matrix = $[[1,0,1], [0,0,0], [1,0,1]]$.
- **Step 4:** రెండు flags false $\rightarrow$ మార్పు లేదు.
- ఫలితం: $[[1,0,1], [0,0,0], [1,0,1]]$. Cascade లేకుండా సరిగ్గా వచ్చింది.

Complexity:

- **Time:** $O(m \times n)$ — క్రిన్సి passes, ప్రతిదీ matrix మీద linear. మొత్తం $O(m \cdot n)$.
- **Space:** $O(1)$ — matrix నే marker storage గా వాడం; రెండు boolean flags మాత్రమే extra. $O(m+n)$ approach ని $O(1)$ కి తగ్గించాం.

గుర్తుంచుకోవాల్సినది: "Grid ని in-place గా, $O(1)$ space లో transform" $\rightarrow$ **matrix నే encoding/marker storage గా reuse చెయ్యి**. ఇక్కడ మొదటి row/col ని markers గా వాడి, వాటిని రెండు flags తో ప్రత్యేకంగా handle చేసే ఉపాయం — చాలా in-place matrix problems కి typical. కీలక సూత్రం: "**చదవడం (mark) పూర్తయ్యాకే రాయడం (apply)**" — cascade అపడానికి రెండు దశలు వేరుగా.

సాధారణ తప్పులు:

- 0 చూసిన వెంటనే row/col ని 0 చేయడం → cascade వల్ల ఎక్కువ cells తప్పుగా 0 (ఈ problem లో #1 తప్పు).
- మొదటి row/col కి flags మర్చిపోవడం → markers వాటి original విలువని పాడుచేసి తప్పు.
- Steps order తప్పు – markers apply (Step 3) చేసి తర్వాత markers set (Step 2) చేయడం, లేదా first row/col (Step 4) ని ముందే 0 చేసి markers పోగొట్టుకోవడం.
- Inner loops ని $r=0, c=0$ నుండి మొదలుపెట్టడం → markers ని data గా తప్పుగా చదవడం. $r=1, c=1$ నుండి.

14. Game of Life (LeetCode #289) – Medium

సమస్య: $m \times n$ board, ప్రతి cell **live (1)** లేదా **dead (0)**. ప్రతి cell కి 8 neighbors (అడ్డం, నిలువు, మూలలు). ఈ నాలుగు rules తో **తర్వాతి state (next generation)** ని లెక్కించు – అన్నీ **ఏకకాలంలో (simultaneously)** జరుగుతాయి:

1. Live cell కి < 2 live neighbors → చనిపోతుంది (underpopulation).
 2. Live cell కి **2 లేదా 3** live neighbors → బతుకుతుంది.
 3. Live cell కి > 3 live neighbors → చనిపోతుంది (overpopulation).
 4. Dead cell కి **సరిగ్గా 3** live neighbors → బతికిపుడుతుంది (reproduction). **In-place** గా update చెయ్యి.
- Constraints: $1 \leq m, n \leq 25$, cells 0 / 1 .

ఉదాహరణ: $\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$.

ఎలా ఆలోచించాలి:

1. **అతిపెద్ద ఉచ్చు – "simultaneously".** ఒక cell ని update చేసేసి, తర్వాత దాని neighbor ని లెక్కిస్తే – ఇప్పటికే మారిన విలువ చదివేస్తాం, తప్పు. అందరూ **అదే క్షణం** మారాలి. అంటే neighbors ని లెక్కించేటప్పుడు ఎప్పుడూ **original** state కావాలి.
2. Naive: board ని copy చేసి, copy నుండి neighbors చదివి, original లో రాయడం. కానీ అది $O(m \cdot n)$ space. Problem in-place అడుగుతోంది.
3. **$O(1)$ insight – ఒకే cell లో రెండు states encode చెయ్యి.** ఒక integer లో **bit 0 = original state, bit 1 = next state**. Neighbors లెక్కించేటప్పుడు $board[nr][nc] \& 1$ (bit 0 = original) చదువు – ఇది మొదటి pass లో ఎప్పుడూ మారదు. Next state ని $| 2$ తో bit 1 లో set చెయ్యి. చివర్లో $>> 1$ చేస్తే next state వస్తుంది.
4. **Direction vectors:** 8 neighbors ని if/else గుట్టులుగా కాకుండా, ఒక **dirs** array $\begin{bmatrix} -1 & -1 \\ \dots \\ 1 & 1 \end{bmatrix}$ తో loop వేసి శుభ్రంగా చూడు.

Brute Force: board యొక్క deep copy తీసుకుని, copy నుండి neighbors చదివి original ని update. Time $O(m \cdot n)$, Space $O(m \cdot n)$. సరళం, కానీ in-place కాదు.

Optimal Approach:

- **Insight:** 2-bit encoding (bit0=old, bit1=new) → original ని పోగొట్టుకోకుండా in-place. $O(1)$ extra space.
- **Pattern:** Matrix traversal – direction vectors + in-place encoding.
- **Plan:** Pass 1 – ప్రతి cell కి live neighbors లెక్కించి, rules ప్రకారం bit 1 set. Pass 2 – అందరినీ $>> 1$.

Solution (JavaScript):

```
function gameOfLife(board) {
  const m = board.length, n = board[0].length;
  // 8 disaila neighbors (moolalu + adda/niluvu)
  const dirs = [[-1,-1],[-1,0],[-1,1],[0,-1],[0,1],[1,-1],[1,0],[1,1]];

  // Pass 1: prati cell ki next state ni bit 1 lo encode
  for (let r = 0; r < m; r++) {
    for (let c = 0; c < n; c++) {
      let live = 0;
      for (const [dr, dc] of dirs) {
        const nr = r + dr, nc = c + dc;
        if (nr >= 0 && nr < m && nc >= 0 && nc < n) {
          live += board[nr][nc] & 1; // bit 0 = ORIGINAL state (marade)
        }
      }
      const alive = board[r][c] & 1;
      // Rule 2 (live baratundi) leda Rule 4 (dead putindi) → bit 1 = 1
      if (alive === 1 && (live === 2 || live === 3)) board[r][c] |= 2;
      else if (alive === 0 && live === 3) board[r][c] |= 2;
      // migata cases: bit 1 = 0 (dead/chastundi) – emi cheyyakkarleda
    }
  }

  // Pass 2: next state (bit 1) ni current ki marchu
  for (let r = 0; r < m; r++) {
    for (let c = 0; c < n; c++) {
      board[r][c] >>= 1;
    }
  }
}

// test
const b = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]];
gameOfLife(b);
console.log(b); // [[0,0,0],[1,0,1],[0,1,1],[0,1,0]]
```

Dry Run: `[[0,1,0],[0,0,1],[1,1,1],[0,0,0]]` (కొన్ని cells)

cell	original	live neighbors	rule	bit1 set?	next
(0,1)	live (1)	1	$< 2 \rightarrow$ చనిపోతుంది	✗	0
(1,0)	dead (0)	3	సరిగ్గా 3 $\rightarrow$ పుడుతుంది	✓	=2`
(1,2)	live (1)	3	3 $\rightarrow$ బతుకుతుంది	✓	1
(2,2)	live (1)	2	2 $\rightarrow$ బతుకుతుంది	✓	1
(3,1)	dead (0)	3	సరిగ్గా 3 $\rightarrow$ పుడుతుంది	✓	1

Pass 1 లో neighbors ని ఎప్పుడూ `& 1` (original bit) తో లెక్కించాం కాబట్టి, పక్క cells మారినా count సరిగ్గా వచ్చింది.

Pass 2 లో `>> 1` $\rightarrow$ ఫలితం `[[0,0,0],[1,0,1],[0,1,1],[0,1,0]]` .

Complexity:

- **Time: $O(m \times n)$** – ప్రతి cell కి 8 neighbors (constant) చూస్తాం $\rightarrow O(8 \cdot m \cdot n) = O(m \cdot n)$. Pass 2 కూడా $O(m \cdot n)$.
- **Space: $O(1)$** – encoding వల్ల board copy అవసరం లేదు; **dirs** constant 8. $O(m \cdot n) \rightarrow O(1)$.

గుర్తుంచుకోవాల్సినది: రెండు కీలక ఆలోచనలు ఒకే problem లో:

- **"అన్నీ ఏకకాలంలో మారాలి" (simultaneous update)** $\rightarrow$ original ని పోగొట్టుకోకుండా, **ఒకే cell లో రెండు states ని encode** (bit0=old, bit1=new; లేదా special codes). చదవడం original నుండి, రాయడం encoded లో.
- **8/4 directions ని **dirs** array తో చూడటం** – grid neighbor problems అన్నిటికీ (DFS/BFS, flood fill) reusable. Interview follow-up: "board infinite అయితే?" – hash set లో live cells మాత్రమే store చేసి, వాటి neighbors ని లెక్కించే sparse approach చెప్పు.

సాధారణ తప్పులు:

- Neighbors ని original కాకుండా updated విలువతో లెక్కించడం (& 1 మర్చిపోవడం) $\rightarrow$ simultaneous rule ఉల్లంఘన, తప్పు next state.
- Pass 2 (>> 1) మర్చిపోవడం $\rightarrow$ board లో 2 , 3 లాంటి encoded విలువలు మిగిలిపోతాయి.
- Boundary check (nr >= 0 && nr < m ...) మర్చిపోవడం $\rightarrow$ array బయటికి / undefined & 1 .
- 8 directions లో కొన్ని (మూలలు) మర్చిపోవడం – diagonal neighbors కూడా లెక్కించాలి.

ముగింపు (Summary) – ఏ pattern ఎప్పుడు?

మూడు patterns ని ఒక్క చూపులో గుర్తుపట్టే cheat-sheet:

సంకేతం (problem లో కనిపిస్తే)	Pattern	ఉప-రకం
Sorted array + pair/triplet sum, palindrome, $O(1)$ space	Two Pointers	opposite-ends
In-place filter, subsequence, "order కాపాడుతూ"	Two Pointers	fast-slow (same-dir)
Maximize area/distance between two indices	Two Pointers	opposite-ends greedy
Contiguous subarray/substring + size k	Sliding Window	fixed
Longest substring/subarray with condition	Sliding Window	variable (invalid అయితే shrink)
Shortest/minimum window covering condition	Sliding Window	variable (valid అయ్యాక shrink)
Grid, spiral, layer-by-layer	Matrix	boundaries (top/bottom/left/right)
Rotate / transpose / diagonal	Matrix	coordinate mapping
Grid + $O(1)$ space in-place transform	Matrix	encoding (markers / bits)
Neighbors / adjacent cells / 8 directions	Matrix	direction vectors

మూడు మంత్రాలు:

1. **Two Pointers** – "రెండే loop బదులు ఇంకో pointer పెట్టొచ్చా?" ($O(n^2) \rightarrow O(n)$). సాధారణంగా order/sorted అవసరం.

2. **Sliding Window** – "అన్ని subarrays చూడాలా, లేక ఒక window ని జారిస్తే చాలా?" **Longest = invalid అయితే shrink; Shortest = valid అయ్యాక shrink.** ప్రతి element ఒకసారి in, ఒకసారి out → $O(n)$.
3. **Matrix** – "boundaries పెట్టొచ్చా? directions array వాడొచ్చా? $O(1)$ కోసం matrix నే encode చేయొచ్చా?"

చివరి మాట: ఈ 14 problems ని కేవలం గుర్తుంచుకోవద్దు – ప్రతిదానిలో "**ఏం గమనించాలి?**" అనే insight ని పట్టుకో. అదే కొత్త problem వచ్చినప్పుడు "ఇది ఏ pattern?" అని గుర్తుపట్టనిస్తుంది. Interview లో pattern గుర్తుపడితే, code దానంతట అదే వస్తుంది. All the best! 🚀